國立成功大學

應用數學研究所

碩士論文

基於差分路徑之降回 Keccak 碰撞攻擊

Collision Attacks on Round-Reduced Keccak based on Differential Trails

研 究 生 ： 吳彥慧

指導教授 ： 黃柏嶧

中華民國一一五年六月

國立成功大學

National Cheng Kung University

碩士論文合格證明書

Certificate of Approval for Master's Thesis

題目：基於差分路徑之降回 Keccak 碰撞攻擊

Title： Collision Attacks on Round-Reduced Keccak based on Differential Trails

研究生：吳彥慧

本論文業經審查及口試合格特此證明

This is to certify that the Master's Thesis of WU YEN HUI

論文考試委員：

has passed the oral defense under the decision of the following committee members

彭宏亭

[illegible]

指導教授 Advisor(s)：黃柏嶸

單位主管 Chair/Director：林敏雄

（單位主管是否簽章授權由各院、系（所、學位學程）自訂）

(The certificate must be signed by the committee members and advisor(s). Each department/graduate institute/degree program can determine whether the chair/director also needs to sign.)

2026/6/5

Abstract

Keccak has been selected by NIST as the SHA-3 hash standard. While the algorithm consists of 24 rounds, practical attacks against the full 24-round Keccak remain infeasible. Consequently, current research focuses on analyzing the security of round-reduced variants. The most effective collision attacks on 5-round Keccak are currently achieved using differential cryptanalysis.

This thesis investigates collision attacks on round-reduced Keccak. We introduce the fundamentals of differential attacks, including both standard differential and internal differential methods. Since the execution of the attack relies on specific differential trails, the initial chapters describe the derivation of a 3-round trail suitable for our attack. Subsequently, we utilize the Target Difference Algorithm (TDA) and Target Internal Difference Algorithm (TIDA) to mount a collision attack on 5-round Keccak. The relevant implementation code is provided in the appendix.

Keywords: Hash Function, SHA-3, Keccak, Differential Cryptanalysis, Collision Attack, Round-Reduced Attack

摘要

Keccak 雜湊函數已被 NIST 選定爲 SHA-3 標準，其演算法包含 24 輪運算。然而，目前學界針對完整 24 輪 Keccak 的攻擊尚未有具體突破，現有的研究主要集中於縮減輪數（Round-Reduced）的安全性分析。目前針對 5 輪 Keccak 最有效的碰撞攻擊，主要採用差分分析法。

本論文旨在探討針對縮減輪數 Keccak 的碰撞攻擊。文中首先介紹差分攻擊的基礎理論，涵蓋一般差分與內部差分（Internal Differential）之概念。由於攻擊的執行仰賴於特定的差分路徑（Differential Trail），本研究前段將詳述如何推導出適用於攻擊的三輪差分路徑。隨後，本研究利用目標差分演算法（Target Difference Algorithm, TDA）與目標內部差分演算法（Target Internal Difference Algorithm, TIDA）等技術，成功實現針對 5 輪 Keccak 的碰撞攻擊。相關的演算法實作程式碼亦收錄於本論文附錄中，以供參考驗證。

關鍵詞：雜湊函數、SHA-3、Keccak、差分攻擊、碰撞攻擊、縮減輪數攻擊

Contents

Abstract i

摘要 ii

1 Introduction 1

2 Preliminaries 4

2.1 Sponge Construction and the SHA-3 Family 4

2.2 The Keccak-f Permutation 5

2.2.1 Inverse of χ 7

3 Differential Trail Search 10

3.1 Differential Cryptanalysis Basics 11

3.1.1 Properties of the θ Mapping 11

3.1.2 Propagation Weight 12

3.1.3 Parity-Bare States 12

3.1.4 The Column-Parity Kernel and Orbitals 13

3.2 Generating the 3-Round Trail 13

3.2.1 Generating the 2-Round Core 14

3.2.2 Trail Extension 17

3.3 Theoretical Lower Bounds for Multi-Round Trails 20

3.4 Trail Search with SAT Solver 21

3.4.1 Input Format of SAT Solvers: Conjunctive Normal Form (CNF) . . 22

3.4.2 Restricting the Propagation Weight of S-boxes Using Auxiliary Variables 22

4 Collision Attack on Round-Reduced Keccak **23**

4.1 Differential Analysis of the Nonlinear Layer 23

4.1.1 Differential Distribution Table (DDT) 24

4.1.2 Values of Difference Conditions Table (VDCT) 24

4.1.3 Probabilistic Linearization and Maximum Difference Density Subspace (MDST) 25

4.2 The Target Difference Algorithm Framework 27

4.2.1 Trail Discovery with SAT Solver 27

4.2.2 Construction of the 2-Round Connector 29

4.2.3 Optimizing Degrees of Freedom: S-box Linearization 31

4.2.4 Extension to the Adaptive 3-Round Connector 33

4.2.5 Collision Search and Two-Block Extension Strategy 33

4.3 The Target Internal Difference Algorithm Framework 34

4.3.1 The Squeeze Attack: A Variant of Birthday Attack 35

4.3.2 Subspace Mapping and Complexity Analysis 36

4.3.3 Theoretical Framework: Internal Symmetry and Overlapping Constraints 37

4.3.4 Message Selection and Free Variable Extraction 39

4.3.5 Message Collection and Subspace Collision Search 40

4.4 Comparative Analysis of TDA and TIDA 41

References **44**

A Inverse of the Linear Layer **47**

A.1 Constructing E_M 50

B DDT Analysis via SAT Solving **52**

B.1 Keccak S-box SAT Solver Modeling 52

B.2 DDT Generation for Keccak χ S-box 52

B.3 Logical Representation of Differential Constraints 56

B.3.1 Analysis of the `attack.cnf` File 56

B.3.2 Encoding Format and Notation 57

B.4 Solver Execution 58

C Target Difference Algorithm (TDA) Implementation **59**

C.1 C++ Implementation of Matrix Inverse and State Generation 59

C.2 Preparation for Finding the 2-Round Connector 64

C.3 2-Round Connector 72

D Target Internal Difference Algorithm (TIDA) **78**

D.1 Finding the Internal Differential Trail via SAT 78

D.2 Implementation of the Message Selection Phase 81

Chapter 1

Introduction

Background and Motivation

Cryptographic hash functions serve as a fundamental building block in modern cybersecurity, providing essential services such as digital signatures, data integrity verification, message authentication codes (MACs), and password storage. An ideal hash function is expected to satisfy three core security properties: preimage resistance, second-preimage resistance, and collision resistance. As computational power continues to evolve and cryptanalytic techniques become more sophisticated, evaluating and ensuring the robustness of these functions remains a critical area of research.

Historically, most widely-standardized hash functions, including MD5, SHA-1, and the SHA-2 family, were designed based on the Merkle-Damgård (M-D) iterative structure. In this framework, a fixed-length compression function is used repeatedly to process input messages of arbitrary length. However, the M-D construction has several inherent theoretical and practical vulnerabilities, most notably its susceptibility to length extension attacks. Furthermore, the groundbreaking collision attacks demonstrated against MD5 and SHA-1 revealed systemic risks within this design paradigm, prompting the cryptographic community to seek more secure alternatives.

In response to these challenges, the U.S. National Institute of Standards and Technology (NIST) initiated the SHA-3 competition in 2007. The Keccak algorithm was eventually selected as the winner and formalized as the SHA-3 standard in 2015. Keccak introduced a radical departure from the M-D structure by employing the innovative ”Sponge Construc-

tion". This design operates in two distinct phases: the absorbing phase, where the message blocks are XORed into the internal state, and the squeezing phase, where the output bits are read from the state.

The sponge-based design of Keccak offers several significant advantages over traditional M-D hash functions:

- **Resistance to Length Extension Attacks**: Due to its large internal state and the nature of the squeezing phase, the sponge construction is inherently immune to the length extension vulnerabilities that plague M-D-based functions.
- **High Flexibility**: By adjusting the ratio between the "rate" and "capacity" parameters, Keccak can be easily tailored to balance security strength against computational performance, while supporting variable-length outputs.
- **Efficiency in Hardware**: The Keccak permutation consists primarily of simple logical operations (XOR, AND, NOT) and bitwise rotations, making it exceptionally efficient for hardware implementations and providing high potential for side-channel resistance.

While Keccak has demonstrated high security, a deeper understanding of its security margin is necessary to evaluate its long-term viability against advanced cryptanalysis. The primary motivation of this research is to analyze the resistance of round-reduced Keccak variants against different classes of attacks, aiming to provide a clearer picture of its security boundaries and offer valuable insights for the deployment of hash functions in security-sensitive applications.

Related Work

Cryptographic hash functions are fundamentally evaluated against three primary security notions: collision resistance, second-preimage resistance, and preimage resistance. A collision attack aims to find any two distinct messages that produce the same hash output. Due to the birthday paradox, this is theoretically the most accessible attack vector, bounded by a generic complexity of $2^{n/2}$ for an n-bit hash function. A second-preimage attack attempts to find a different message that hashes to the same output as a specific, given input. Finally, a preimage attack seeks to recover an original input message from a given target hash value.

Preimage recovery is theoretically the most challenging attack, requiring an exhaustive search complexity of 2^n.

To efficiently bypass this generic bound on round-reduced Keccak, several advanced methodologies have been developed in the literature, including cross-linear structures [4], the allocating approach [2], and recent internal differential techniques targeting up to 5 rounds [3]. In this thesis, we evaluate the security margin of round-reduced Keccak. Specifically, we achieve a successful 5-round collision attack using differential cryptanalysis, and a 4-round preimage attack utilizing the allocating approach based on linear structures. This discrepancy in the attacked rounds clearly reflects the inherent theoretical hardness between finding collisions and preimages.

Thesis Organization

The remainder of this thesis is organized as follows:

In **Chapter 2**, we provide a comprehensive overview of the SHA-3 standard, including the foundational sponge construction and the five component mappings of the Keccak-f permutation. Special emphasis is placed on the inversion of the linear layer—detailed in the corresponding Appendix—as it is critical for the cryptanalysis presented in subsequent chapters. Additionally, we present the derivation for the inverse of the non-linear χ operation.

Chapter 3 focuses on the concept of differential trails, which serve as the preliminary step for both the Target Difference Algorithm (TDA) and the Target Internal Difference Algorithm (TIDA). By analyzing the trail space, we demonstrate the impossibility of finding a 5-round differential trail with a sufficiently high probability to facilitate a collision attack. To address the complexity of trail searching, the final section of this chapter explores the implementation of SAT solvers as an efficient alternative for identifying valid trails.

Chapter 4 introduces the methodologies for finding collisions: the TDA and TIDA. We provide a detailed narration of the TDA process, specifically the identification of differential trails and the construction of 2-round connectors, followed by an extension to 3-round connectors. Regarding the TIDA, we focus on its distinct features and procedural differences compared to the TDA to ensure conciseness. This chapter concludes with a comparative analysis of the performance and characteristics of these two algorithms.

Chapter 2

Preliminaries

2.1 Sponge Construction and the SHA-3 Family

The sponge construction, primarily detailed in the SHA-3 standard [1], is a framework used to define functions on binary data with arbitrary output lengths. It operates on a fixed-length internal state of b bits, which is partitioned into two components: the rate r and the capacity c, such that $b = r + c$. The evaluation of a sponge function consists of two primary phases: the absorbing phase and the squeezing phase.

Before the absorbing phase begins, the input message must be padded to ensure its length becomes a positive multiple of the rate r. KECCAK employs a multi-rate padding rule denoted as `pad10*1`. This rule appends a single bit 1, followed by a possibly empty sequence of 0s, and ends with another bit 1. The exact number of 0s is determined such that the total length of the padded message is a multiple of r.

Furthermore, to support domain separation, specific suffixes are appended to the input message prior to the application of the `pad10*1` padding rule. The four standard SHA-3 hash functions (SHA3-224, SHA3-256, SHA3-384, and SHA3-512) append a two-bit suffix `01` to the message. In contrast, the SHA-3 Extendable-Output Functions (XOFs), specifically SHAKE128 and SHAKE256, append a four-bit suffix `1111` to distinguish their application domain.

During the absorbing phase, the padded input is divided into blocks of r bits. Each r-bit block is sequentially XORed into the first r bits of the internal state, while the remaining c bits (the capacity) are left unmodified during this injection step. Immediately after each

XOR operation, the underlying permutation function f (which is KECCAK-$p[1600, 24]$ for the SHA-3 family, denoting a 1600-bit state operated over 24 rounds) is applied to the entire b-bit state, thereby diffusing the absorbed input across both the rate and the capacity.

Once all input blocks have been absorbed, the construction enters the squeezing phase. In this phase, the first r bits of the state are extracted and returned as the output. If additional output bits are required to meet the target digest length d, the permutation f is iteratively applied to the state, and the next r bits are extracted each time. This process is repeated until a sufficient number of bits have been generated, at which point the final output string is truncated to exactly d bits.

Based on this framework, the SHA-3 family defines specific parameter sets for different security requirements, all utilizing the fixed state size of $b = 1600$ bits. For the four fixed-length hash functions, the target digest length d is predefined ($d \in \{224, 256, 384, 512\}$), and the capacity is strictly set to $c = 2d$ to provide the corresponding security level. Consequently, the rate r is determined by $1600 - 2d$.

On the other hand, the Extendable-Output Functions (SHAKE128 and SHAKE256) provide the flexibility of an arbitrary output length d, which is specified by the user. Instead of fixing d, the SHAKE functions fix their capacities to ensure a specific security strength against cryptographic attacks: SHAKE128 operates with a capacity $c = 256$ (rate $r = 1344$), while SHAKE256 operates with $c = 512$ (rate $r = 1088$).

2.2 The Keccak-f Permutation

The Keccak-f permutation consists of 24 rounds, and each round function R is composed of five sequential step mappings: $\theta, \rho, \pi, \chi, \iota$. The state is denoted as a 3-dimensional array $A[x, y, z]$, where $x, y \in \{0 \dots 4\}$ and $z \in \{0 \dots 63\}$. For the standard Keccak-$f[1600]$ permutation, the lane size is strictly fixed at 64 bits, yielding a total state size of $5 \times 5 \times 64 = 1600$ bits. The detailed definitions of these steps are described as follows:

θ **(Theta):** This step serves as the linear diffusion layer. It computes the parity of two columns adjacent to the target bit and adds it to the state. The operation provides

high diffusion and is defined as:

$$A'[x, y, z] = A[x, y, z] \oplus \sum_{j=0}^{4} A[x-1, j, z] \oplus \sum_{j=0}^{4} A[x+1, j, z-1]. \tag{2.1}$$

ρ **(Rho):** This step represents the translation (rotation) layer. It rotates the bits of each lane along the z-axis by a specific offset $r[x, y]$, providing inter-slice dispersion:

$$A'[x, y, z] = A[x, y, z - r[x, y] \pmod{64}]. \tag{2.2}$$

π **(Pi):** This step is a permutation that rearranges the positions of the lanes within the state matrix. It disrupts the column alignment and is defined by the linear transformation:

$$A'[y, 2x + 3y, z] = A[x, y, z]. \tag{2.3}$$

χ **(Chi):** This is the only non-linear mapping in the Keccak round function. It can be viewed as applying a 5-bit S-box to each row. The operation is defined as:

$$A'[x, y, z] = A[x, y, z] \oplus ((\neg A[x+1, y, z]) \wedge A[x+2, y, z]). \tag{2.4}$$

The algebraic degree of χ is 2.

ι **(Iota):** This step adds a round-dependent constant to break the symmetry across different rounds of the permutation. It only affects the lane at the origin $(0, 0)$. Let $i_r \in \{0 \dots 23\}$ denote the round index; the operation for round i_r is defined as:

$$A'[0, 0, z] = A[0, 0, z] \oplus RC[i_r][z], \tag{2.5}$$

where $RC[i_r]$ is the specific round constant configured for round i_r, and $RC[i_r][z]$ represents the z-th bit of this constant.

The inverses of θ and π are in Appendix A.

Note on Differential Analysis:

Among the five steps, θ, ρ, and π are linear layers, while χ provides the non-linearity required

to resist algebraic attacks. Regarding differential cryptanalysis, since ι involves XORing a fixed constant, it does not affect the propagation of XOR differences ($\Delta x \oplus \Delta c = \Delta x$). Therefore, ι is typically omitted (or ignored) when searching for differential trails.

2.2.1 Inverse of χ

The derivation in this section primarily follows the approach presented in [15]. Let the state of the row be 5 bits, where the indices are evaluated modulo 5. The χ mapping in Keccak is defined as:

$$y_i = x_i \oplus (x_{i+1} \oplus 1)x_{i+2} \quad \text{for } i = 0, \ldots, 4 \tag{2.6}$$

To deduce the inverse mapping $\chi^{-1} : y \to x$, we first multiply both sides of the equation by x_{i+1}:

$$y_i x_{i+1} = x_i x_{i+1} \oplus (x_{i+1} \oplus 1)x_{i+2}x_{i+1}$$

Since $x_{i+1}(x_{i+1} \oplus 1) = 0$ in $\mathbb{F}_2$, the second term vanishes, yielding:

$$y_i x_{i+1} = x_i x_{i+1} \tag{2.7}$$

By shifting the indices ($i \to i+1$), we obtain $y_{i+1}x_{i+2} = x_{i+1}x_{i+2}$, which can be rewritten as $(y_{i+1} \oplus 1)x_{i+2} = (x_{i+1} \oplus 1)x_{i+2}$. Substituting this property back into the original definition of χ, we get:

$$\begin{aligned} y_i &= x_i \oplus (y_{i+1} \oplus 1)x_{i+2} \\ \implies x_i &= y_i \oplus (y_{i+1} \oplus 1)x_{i+2} \end{aligned} \tag{2.8}$$

By shifting the indices in Equation 2.8 ($i \to i + 2$), we can express x_{i+2} as:

$$x_{i+2} = y_{i+2} \oplus (y_{i+3} \oplus 1)x_{i+4} \tag{2.9}$$

Substituting Equation 2.9 into Equation 2.8 gives:

$$\begin{aligned} x_i &= y_i \oplus (y_{i+1} \oplus 1)\Big(y_{i+2} \oplus (y_{i+3} \oplus 1)x_{i+4}\Big) \\ &= y_i \oplus (y_{i+1} \oplus 1)y_{i+2} \oplus (y_{i+1} \oplus 1)(y_{i+3} \oplus 1)x_{i+4} \end{aligned} \tag{2.10}$$

To complete the proof, we must prove that x_{i+4} in the last term can be replaced by y_{i+4}. Recall that the indices are evaluated modulo 5, so $y_{i+4} = x_{i+4} \oplus (x_{i+5} \oplus 1)x_{i+6} = x_{i+4} \oplus (x_i \oplus 1)x_{i+1}$. Thus:

$$x_{i+4} = y_{i+4} \oplus (x_i \oplus 1)x_{i+1}$$

From our earlier deduction, we know $(x_i \oplus 1)x_{i+1} = (y_i \oplus 1)x_{i+1}$. Therefore, $x_{i+4} = y_{i+4} \oplus (y_i \oplus 1)x_{i+1}$. Substituting this into the last term of Equation 2.10:

$$\begin{aligned} (y_{i+1} \oplus 1)(y_{i+3} \oplus 1)x_{i+4} &= (y_{i+1} \oplus 1)(y_{i+3} \oplus 1)\Big(y_{i+4} \oplus (y_i \oplus 1)x_{i+1}\Big) \\ &= (y_{i+1} \oplus 1)(y_{i+3} \oplus 1)y_{i+4} \oplus \Delta \end{aligned}$$

where the difference $\Delta = (y_i \oplus 1)(y_{i+1} \oplus 1)(y_{i+3} \oplus 1)x_{i+1}$. We now demonstrate that $\Delta = 0$ unconditionally. Observe the dependencies of the terms:

- $(y_i \oplus 1)x_{i+1} = (x_i \oplus 1)x_{i+1}$. If this evaluates to 1, then $x_{i+1} = 1$.
- Given $x_{i+1} = 1$, we have $y_{i+1} = x_{i+1} \oplus (x_{i+2} \oplus 1)x_{i+3} = 1 \oplus (x_{i+2} \oplus 1)x_{i+3}$. Thus, $(y_{i+1} \oplus 1) = (x_{i+2} \oplus 1)x_{i+3}$. For this to be 1, x_{i+3} must be 1.
- Given $x_{i+3} = 1$, we similarly find that $(y_{i+3} \oplus 1) = (x_{i+4} \oplus 1)x_{i+5} = (x_{i+4} \oplus 1)x_i$.

Combining these implications, the product evaluates as:

$$\Delta = (x_i \oplus 1)x_{i+1} \cdot (x_{i+2} \oplus 1)x_{i+3} \cdot (x_{i+4} \oplus 1)x_i$$

The presence of both $(x_i \oplus 1)$ and x_i in the boolean product guarantees that $\Delta = 0$ since $x_i(x_i \oplus 1) = 0$ in $\mathbb{F}_2$.

Consequently, $(y_{i+1} \oplus 1)(y_{i+3} \oplus 1)x_{i+4} = (y_{i+1} \oplus 1)(y_{i+3} \oplus 1)y_{i+4}$. The inverse of χ is definitively formulated as:

$$x_i = y_i \oplus ((y_{i+1} \oplus 1)y_{i+2}) \oplus ((y_{i+1} \oplus 1)(y_{i+3} \oplus 1)y_{i+4}) \tag{2.11}$$

which is logically equivalent to $x_i = y_i \oplus (y_{i+1} \oplus 1) \cdot \Big(y_{i+2} \oplus (y_{i+3} \oplus 1) y_{i+4}\Big)$. This completes the proof.

Chapter 3

Differential Trail Search

In this chapter, we detail the methodology employed for our differential trail search. We commence by exhaustively traversing the space of 3-round trail cores up to a computationally feasible weight limit. This enumeration allows us to deduce theoretical lower bounds for trails spanning four rounds or more.

Although the established theoretical lower bound for a 5-round trail does not appear prohibitively large, it is crucial to recognize that this bound is not tight. It merely represents a strict mathematical floor, and the true minimum weight of an existing continuous trail is likely to be substantially higher. In practice, the specific differential paths we are able to discover possess propagation weights that are far too large for a straightforward differential attack. These excessively high-weight trails would consume all available degrees of freedom, making the direct application of a continuous 5- or 6-round trail practically infeasible for finding collisions.

To overcome this critical limitation, we introduce a connector strategy in the next chapter, which allows us to bridge intermediate states without relying on a single, high-weight trail. Finally, we formulate the path-finding problem algebraically and leverage a SAT-based automated search toolkit to efficiently construct the exact differential paths required for the complete attack.

3.1 Differential Cryptanalysis Basics

To simplify the analysis of the differential propagation, we consider the state bits in the context of the "in-kernel" and "out-kernel" of the θ mapping [19][21][22]. These concepts are fundamental in determining the propagation of patterns through the Keccak-f rounds.

3.1.1 Properties of the θ Mapping

The details of the following properties are written in [19]. The mapping θ is the main linear layer providing diffusion. A column, denoted as $C[x][z]$, is defined as the parity of the five bits within that column.

- **θ-Effect:** We define a column $C[x][z]$ as being *θ-affected* if its resulting value after the transformation is influenced by the parities of its neighboring columns, specifically from the positions $(x-1, z)$ and $(x+1, z-1)$.
- **State Projection:** The 1600-bit state can be projected onto a 320-bit parity plane by XORing the bits in each column. This projection allows us to focus on the propagation of column parities rather than individual bits.

Linear Representation and Sequence Mapping

To facilitate the search for low-weight trails, we express the θ transformation using the polynomial or coordinate offsets $(-t, 2t)$. By mapping the bits $B[x][z]$ of the parity plane into a one-dimensional sequence according to these offsets, we can observe the diffusion characteristics more clearly.

In this one-dimensional representation, the occurrence of a θ-affected column is manifested as a transition (a change from 0 to 1 or vice versa) within the sequence. This approach is particularly advantageous for counting the number of active θ-affected columns, as it reduces the problem of weight evaluation to a sequence analysis problem.

One-Dimensional Sequence Representation

By applying a specific mapping, such as using the offsets $(-1, 0)$ and $(1, 1)$, we can linearize the 320-bit parity plane into a one-dimensional sequence. In this representation, a θ-affected

column corresponds to a transition or a ”change” in the parity sequence (e.g., a transition from 0 to 1). This abstraction makes it significantly more convenient to count the number of active θ-affected columns and evaluate the differential weight of a given characteristic.

3.1.2 Propagation Weight

Following the framework introduced in [19], the propagation weight $\omega(u \to v)$ of a differential transition through the non-linear mapping χ is defined as the negative binary logarithm of its differential probability:

$$\omega(u \to v) = -\log_2 \left(\Pr[\chi(A \oplus u) \oplus \chi(A) = v]\right) \tag{3.1}$$

Because χ operates on 5-bit rows independently, the overall weight $\omega(u \to v)$ is simply the sum of the weights across all active rows. Consequently, for a 3-round trail where (α_i, β_i) denote the input and output differences of the i-th round, the total weight is accumulated as follows:

$$W_{total} = \omega(\beta_0 \to \alpha_1) + \omega(\beta_1 \to \alpha_2) + \omega(\beta_2 \to \alpha_3) \tag{3.2}$$

In the context of our attack, we evaluate transitions between a strictly fixed input difference u and a target output difference v. The propagation weight directly dictates the size of the valid solution space: there are exactly $2^{n-\omega(u \to v)}$ valid n-bit state values A that permit this exact transition. This inverse exponential relationship illustrates that a higher propagation weight translates into fewer valid state solutions, thereby imposing stricter algebraic conditions and heavily consuming the available degrees of freedom required for the collision search.

3.1.3 Parity-Bare States

To analyze the propagation of differential trails with minimal weight, we utilize the concept of *parity-bare states* introduced by Mella et al. [21]. Given a 320-bit parity plane $P[x][z]$, where each bit represents the XOR sum of a column in the 1600-bit state A, we can construct a representative state A_{bare} that satisfies the parity conditions while minimizing the active bit count.

The construction of the parity-bare states follows these rules:

- **Active Columns:** If the column sum is 1 ($C[x][z] = 1$), we assign exactly one bit in that column to be 1 (typically $A[x][0][z] = 1$) and the remaining four bits to be 0.
- **Inactive Columns:** If the column sum is 0 ($C[x][z] = 0$), all five bits in that column are set to 0.

By defining the state in this manner, we create a one-to-one mapping between the 320-bit parity plane and a sparse 1600-bit state. This ”bare” representation serves as a baseline for calculating the lower bound of the differential weight before considering the effects of ρ and π transformations.

3.1.4 The Column-Parity Kernel and Orbitals

In the differential cryptanalysis of Keccak, analyzing states that reside in the column-parity kernel (i.e., states where the θ mapping acts as the identity) is crucial. The fundamental building block of these in-kernel states is the *orbital*.

An orbital is defined as the simplest possible state configuration that belongs to the column-parity kernel. It consists of exactly two active bits located in the same column—meaning they share the same x and z coordinates but have different y coordinates. Because the θ step computes the parity of each column, the XOR sum of these two active bits evaluates to zero over $\mathbb{F}_2$. Consequently, an orbital passes through the θ layer without triggering any diffusion (i.e., avoiding the θ-effect).

By linearly combining multiple disjoint orbitals, an attacker can construct more complex in-kernel states (such as those required for $|K|K|$ and $|K|N|$ trail cores) while maintaining strict control over the total propagation weight.

3.2 Generating the 3-Round Trail

The construction of a 3-round differential trail is performed using a two-phase strategy. The process begins with the identification of a valid 2-round differential sequence, commonly referred to in the literature as the *core* (or 3-round trail core) [22]. This core is required to exhibit a low Hamming weight and a high propagation probability. Once all viable 2-round

cores are generated, they are subsequently extended either forward to a compatible state S_3 or backward by one additional round to form the complete 3-round trail. This extension process utilizes the techniques from [21] to ensure that the resulting 3-round sequence remains cryptographically significant.

3.2.1 Generating the 2-Round Core

We define the 2-round core as the differential sequence $\alpha_1 \rightarrow \beta_1 \rightarrow \alpha_2 \rightarrow \beta_2$. Based on whether the state differences α_1 and α_2 belong to the Column-Parity Kernel (K) or not (N), we can classify these 2-round cores into four distinct categories: $|K|K|$, $|K|N|$, $|N|K|$, and $|N|N|$.

$|K|K|$ Cores

In a $|K|K|$ core, both the first and second round input differences (α_1 and α_2) must reside in the Column-Parity Kernel. This implies that prior to the θ operation, the sum of every column over $\mathbb{F}_2$ is zero. Consequently, the θ operation introduces no new active bits and acts purely as an identity function.

To maintain the in-Kernel property for α_2, any active slice in β_1 must contain at least two active bits. This minimum threshold ensures that the non-linear χ operation can produce an even number of active bits per affected column, which is strictly required to satisfy the parity condition for α_2. For instance, if α_1 consists of 3 orbitals (equivalent to 6 active bits), these bits are distributed across β_1 slices via the $\pi \circ \rho$ transformation. Due to the diffusion properties of $\pi \circ \rho$, an active β_1 slice can contain at most m bits, where m is the number of orbitals in α_1. Therefore, for $m = 3$, the only valid candidate β_1 structures—denoted by the number of active bits in each slice—are $\{3, 3\}$ and $\{2, 2, 2\}$.

$|N|K|$ and $|N|N|$ Cores

In this study, our primary objective is to exhaustively generate all valid 2-round cores that can potentially be extended into 3-round trails with a propagation weight no greater than a predefined threshold w. To effectively prune the massive search space during the 2-round generation phase, we employ a look-ahead bounding strategy. In practice, the propagation

weight is heavily dependent on the number of active rows. Therefore, we estimate the theoretical minimum active rows for the eventual 3-round trail, denoted as Row_3. The total active rows are defined by the equation:

$$Row_3 = Row_{\alpha_1} + Row_{\beta_1} + Row_{\beta_2}$$

where each component explicitly represents the number of active rows at specific stages of the trail:

- Row_{α_1}: The number of active rows in the state difference α_1, which is the output difference of the preceding χ layer.
- Row_{β_1}: The number of active rows in the difference β_1, which is the output difference of the first linear layer (and the input to the subsequent χ layer).
- Row_{β_2}: The number of active rows in the difference β_2, which is the output difference of the second linear layer.

To systematically cover the search space, we group states sharing the same parity into a derivative subspace V_{α_1}, generated from a corresponding parity-bare state. To efficiently evaluate whether a subspace V_{α_1} contains any valid cores, we construct a **theoretical representative** (denoted as α_1^r) for each subspace based on the **Ideal Improvement Assumption**. Instead of exhaustively testing every combination, this assumption postulates that the original parity-bare state can be optimally improved to minimize the anticipated 3-round total Row_3. This is achieved by ideally processing the three components of Row_3 as follows:

1. **Ideal β_1 for optimal Row_{β_2}:** We assume each β_1 slice that lacks any in-Kernel χ-compatible slices is optimally transformed to possess at least one in-Kernel χ-compatible slice by adding only one active bit to it. Furthermore, we assume the other bit of this added orbital optimally improves another such slice simultaneously. This assumption yields an optimal (minimal) Row_{β_2} without introducing any new β_1 slices, utilizing the least number of orbitals.
2. **Counting Row_{β_1} with the least row increase:** While there are multiple ways to improve a slice (e.g., using bits from active or inactive rows), we strictly select the im-

provement strategy that results in the minimal increase to the total number of active rows in β_1.

3. **Counting Row_{α_1} with the least row increase:** When calculating the new Row_{α_1} after the β_1 improvements, we consistently choose the combination of added orbitals that incurs the absolute minimum row increase at α_1.

The optimal state synthesized under these three rigorous conditions is defined as the theoretical representative α_1^r of the derivative subspace V_{α_1}.

While this strictly theoretical representative might not exist in practice due to the rigid bit-placement constraints of the $\pi \circ \rho$ linear operations, it successfully establishes an absolute lower bound for the propagation weight of any state within V_{α_1}. Consequently, we subject this theoretical representative to a **viability check**. If the theoretically optimal Row_3 of α_1^r exceeds our target weight threshold, it mathematically guarantees that no valid trail can be derived from this entire subspace, allowing it to be safely discarded. Conversely, viable subspaces are systematically explored by iteratively adding orbitals to the viable states, ensuring an exhaustive collection of all valid 2-round cores.

$|K|N|$ Cores

The methodology for searching 2-round cores with the $|K|N|$ characteristic shares the same fundamental strategy as the $|N|K|$ and $|N|N|$ searches. The core concept remains the viability check: the search space of all out-Kernel states α_2 is partitioned into a series of derivative subspaces V_{α_2}, represented by parity-bare states under candidate parities. Subsequently, the ideal improvement assumption is applied to construct theoretical representatives for these subspaces, allowing us to efficiently prune the search space by evaluating their required number of active rows.

However, the defining distinction of the $|K|N|$ case is the strict requirement that α_1 must be in-Kernel. As a result, the criteria for the ideal improvement assumption must be specifically adapted. Under the analytical model $\alpha_1 \xleftarrow{\lambda'^{-1}} \beta_1 \xleftarrow{\chi^{-1}} \alpha_2 \xrightarrow{\lambda} \beta_2$ (where $\lambda'^{-1} = \rho^{-1} \circ \pi^{-1}$), we classify the active rows of β_1 into *orbital rows* and *non-orbital rows*. To guarantee that α_1 is in-Kernel with the minimal propagation cost, we introduce the **ideal compensating assumption**.

Instead of merely transforming common slices into preferred ones as in the $|N|K|$ case, the $|K|N|$ search focuses on compensating non-orbital β_1 rows. We assume that the necessary orbitals are added to α_2 to convert all non-orbital β_1 rows into orbital rows, thereby ensuring the resulting α_1 states consist entirely of valid orbitals. The theoretical representative of V_{α_2} is constructed by choosing the combination of added α_2 orbitals that incurs the least possible increase to the look-ahead active row count $Row_3 = Row_{\alpha_1} + Row_{\alpha_2} + Row_{\beta_2}$.

Once the theoretical representative is established, the viability check is conducted by verifying if the estimated $Row_3 \leq \lceil \frac{T_3}{2} \rceil$. For each viable subspace, we iteratively add one orbital at a time to the original α_2 and perform further viability checks. Finally, all collected viable α_2 states are extended backward by one round to generate the complete set of valid 2-round cores with the $|K|N|$ feature.

3.2.2 Trail Extension

Once the foundational 2-round cores are exhaustively generated and filtered using the look-ahead bounding strategy, the second phase of the algorithm involves physically extending these cores to reach the target length of three rounds. Depending on the properties of the core, this extension can be performed either backward or forward.

For a backward extension, the state α_1 is propagated inversely through the round function to determine a compatible initial state β_0. Conversely, in a forward extension, β_2 is propagated forward to compute the subsequent state α_3. During this phase, the theoretical weight bounds (Row_3) that were estimated and used for pruning during the core generation are formally realized. The actual propagation weight of the newly formed 3-round trail is calculated and verified against the target threshold. Ultimately, this extension process converts the optimized 2-round cores into complete, cryptographically significant 3-round differential trails ready for the collision attack.

Forward Extension

The forward extension process is categorized into two distinct cases: extension inside the kernel and extension outside the kernel, each requiring a tailored algorithmic approach [21]. To contextualize this process, consider a known differential trail for the current round, denoted as $a_1 \xrightarrow{\lambda} b_1$. The objective is to propagate the output state b_1 through the non-linear layer χ

to derive the input state a_2 for the next round. Crucially, the designation of being "inside the kernel" or "outside the kernel" refers exclusively to the properties of this newly generated target state a_2.

Given b_1, the set of all χ-compatible patterns a_2 forms an affine space $U(b_1) = V + e$. To determine if this space intersects with the kernel ($a_2 \in K$), the algorithm evaluates column parities slice by slice. If the offset's parity e_p falls within the span of the basis parities V_p (i.e., $e_p[z] \in \text{Span}(V_p[z])$), a valid in-kernel offset e' can be constructed. Since the resulting in-kernel affine subspace $e' + V_K$ typically has an extremely low dimension, all valid extensions a_2 inside the kernel are generated through direct exhaustive enumeration, bypassing the need for complex search trees.

In contrast, forward extension outside the kernel involves a much larger search space, requiring the construction of a search tree rooted at the offset e. The elements a_2 are generated by iteratively adding basis vectors from the set $V = V_K + V_N$. While the goal is to find an outside-kernel state, the search tree utilizes both in-kernel (V_K) and outside-kernel (V_N) basis vectors as building blocks. Because the weight function is non-monotonic, we must define the worst-case weight loss for every type of basis vector to enable efficient pruning of the tree:

- **In-kernel basis vectors (V_K):** For a vector containing n orbitals, the maximum weight loss is bounded by $4n$. An orbital that does not overlap with the current state a_2 incurs no loss, while an overlapping orbital loses at most 2.
- **Outside-kernel basis vectors (V_N):** For a vector with Hamming weight q, if its affected columns overlap with n affected columns of a_2, the weight loss is strictly bounded by $10n + 2q$.

The derivation of the $10n + 2q$ bound stems from the structural properties of Keccak. In the χ layer, each active S-box contributes a minimum weight of 2; thus, canceling one active bit leads to a maximum weight loss of 2. The $10n$ term represents the potential cancellation of the θ-effect: since θ diffuses parities across all 5 rows of a column, canceling one affected column removes up to 5 active bits, resulting in a weight loss of $5 \times 2 = 10$. The $2q$ term accounts for the q primitive bits of the basis vector itself, which may each incur a weight loss of 2 if canceled by bits in a_2.

By subtracting these maximum potential losses from the current weight, the algorithm can safely prune any subtree that is mathematically guaranteed to exceed the target weight limit, thereby significantly optimizing the search.

Backward Extension

The theoretical foundation and algorithmic procedures for the backward extension detailed in this section are primarily derived from the framework introduced by [21].

Consider the backward extension process, where a known state is propagated backward through the non-linear layer χ^{-1}, such as inferring b_1 from a_2, or b_0 from a_1. Unlike forward extension, the set of all patterns compatible through χ^{-1} does not strictly form an affine space. To efficiently scan and evaluate these patterns, we must construct a larger affine space that completely encompasses all such valid patterns, referred to as the χ^{-1}-envelope space.

The offset and basis vectors of this χ^{-1}-envelope space are generated on a row-by-row basis. The construction strategy is categorized into two distinct cases depending on the number of active bits in the target row:

- **Single active bit in a row:** Based on the algebraic properties of the χ mapping, if an active row contains exactly one active bit at position (x, y, z), the corresponding bit in any valid χ^{-1}-compatible state must also remain active. Therefore, we set the offset for this row to have a single active bit at the identical position (x, y, z), and we construct four basis vectors, each containing a single active bit at the remaining positions (x', y, z) where $x' \neq x$.

- **Multiple active bits in a row:** If the target active row contains more than one active bit, we do not assign a specific offset constraint. Instead, we expand the space by utilizing a full 5-dimensional vector space. In this case, we generate five basis vectors, each containing exactly one single active bit, to ensure all possible valid configurations for that row are covered during the tree traversal.

Similar to forward extension, we must determine whether the backward extension yields a state inside or outside the parity kernel. To evaluate kernel membership, we check if the parity of the offset can be expressed as a linear combination of the parities of the basis vectors.

Extensions restricted to the kernel are computationally straightforward and can be evaluated directly.

For backward extensions outside the kernel, we limit the search space by analyzing the parity patterns. The state is projected into a parity pattern by summing the bits along the y-axis (columns). Let p' denote the parity pattern before the θ mapping, and p denote the parity pattern after θ. The effect of the θ step yields the relationship:

$$p[x, z] = p'[x, z] + p'[x - 1, z] + p'[x + 1, z - 1] \tag{3.3}$$

By rearranging this equation, we obtain a recurrence relation that allows us to deduce the parity of the previous slice $z - 1$:

$$p'[x + 1, z - 1] = p[x, z] + p'[x, z] + p'[x - 1, z] \tag{3.4}$$

Using this relation, we can fully reconstruct p' by making an initial assumption for the row at slice $z = 0$. From this guess, we iteratively deduce the slices at $z = w - 1, w - 2, \ldots$ until we loop back to $z = 0$. If the newly computed parity at $z = 0$ matches the initial assumption, the guess is valid, and the resulting parity pattern is consistent. By exhaustively testing all 32 possible values for the initial row guess, we can systematically isolate all valid parity patterns.

3.3 Theoretical Lower Bounds for Multi-Round Trails

Following the advanced exhaustive differential trail search techniques introduced by Liu et al. [22], the search space of 3-round differential trail cores in KECCAK-$f[1600]$ has been successfully extended to cover propagation weights up to $T_3 = 53$ (up from the previous best threshold of 45). By leveraging the comprehensive knowledge of all 3-round trail cores within this expanded weight range, the theoretical lower bounds for differential trails spanning 4, 5, and 6 rounds have been significantly improved .

Specifically, while the minimum propagation weights for 1-round and 2-round trails remain trivially bounded at 2 and 8 respectively, the accumulated weight over subsequent rounds escalates rapidly. Based on the complete enumeration of 3-round trail cores up to

weight 53, the updated lower bounds on multi-round propagation weights are deduced as follows:

- **4 Rounds:** The theoretical lower bound of the propagation weight is raised to 56.
- **5 Rounds:** The theoretical lower bound of the propagation weight is raised to 58.
- **6 Rounds:** The theoretical lower bound of the propagation weight is raised to 108.

In standard differential cryptanalysis, the propagation weight w is defined as the negative binary logarithm of the differential probability, meaning that following a specific trail implies a probability of $p = 2^{-w}$[19]. While searching for a differential trail with a weight of 58 is computationally feasible (requiring $\approx 2^{58}$ operations), it is crucial to recognize that this value merely represents a strict mathematical lower bound. Currently, no actual 5-round differential trail matching this exact minimal weight has been discovered; we have only proven the non-existence of trails with weights strictly below this threshold.

Consequently, relying exclusively on a full-round, contiguous differential characteristic to mount a collision attack remains highly impractical, as the actual minimum weight of an existing, coherent trail is likely much higher. To successfully bridge this gap and construct practical collisions, advanced cryptanalytic frameworks are required. Attackers must incorporate dedicated techniques—such as utilizing multi-round connectors to efficiently link intermediate round differences, or translating the algebraic system into Conjunctive Normal Form (CNF) to be resolved automatically via SAT solver.

3.4 Trail Search with SAT Solver

As discussed in the previous section, while the trail enumeration method can exhaustively find all differential trails with a probability above a certain threshold, it remains computationally inefficient. Since our current objective is no longer to exhaustively enumerate all possibilities, but rather to find specific trails that satisfy a set of predefined conditions (namely, spanning four forward rounds and two backward rounds), this search is inherently a constraint satisfaction problem. Therefore, we employ a SAT solver, which is highly efficient at discovering trails that meet these exact structural requirements without exploring the entire search space.

These trails are subsequently linked using a connector strategy—which will be detailed in the following chapter—to significantly accelerate the overall collision search process.

3.4.1 Input Format of SAT Solvers: Conjunctive Normal Form (CNF)

The standard input format for modern cryptographic SAT solver, such as CryptoMiniSat [16], is the Conjunctive Normal Form (CNF). A CNF formula is defined as a conjunction (logical AND) of one or more clauses. Each clause, in turn, is a disjunction (logical OR) of literals. In the standard DIMACS format commonly utilized by SAT solvers, these literals are represented by non-zero integers, where positive and negative integers denote the corresponding Boolean variables and their negations, respectively. Furthermore, each clause is strictly terminated by a `0`. For further details regarding the CNF encoding, please refer to Appendix B.2.

3.4.2 Restricting the Propagation Weight of S-boxes Using Auxiliary Variables

To restrict the upper bound of the number of active S-boxes, it is necessary to introduce auxiliary variables. Specifically, we employ the sequential counter encoding for Boolean cardinality constraints proposed by Sinz [17]. Let $x_k \in \{0, 1\}$ be a boolean variable indicating whether the k-th S-box is active, and let W be the predefined maximum weight limit. We introduce a set of binary auxiliary variables $s_{i,j}$ such that $s_{i,j} = 1$ if and only if $\sum_{k=0}^{i} x_k \geq j$.

While this straightforward formulation initially yields $n(W + 1)$ auxiliary variables—where n is the total number of S-boxes involved across the target rounds—we observe that $s_{0,j}$ is inherently equal to zero (assuming $j \geq 1$). By eliminating these W constant variables, the total number of required auxiliary variables is effectively reduced to $n(W + 1) - W$.

Chapter 4

Collision Attack on Round-Reduced Keccak

In this section, we present a collision attack on the 5-round reduced Keccak permutation utilizing differential cryptanalysis. Whether considering external differentials between two distinct messages or tracking internal differential propagation through the state, several common properties emerge that facilitate the attack.

A primary component of our strategy involves the linearization of the non-linear mapping χ. By exploiting specific properties of the Keccak-f round function—specifically the interaction between the linear layers and the algebraic degree of the S-box—we identify high-probability differential paths. These linear approximations allow for the construction of a system of equations whose solutions yield colliding message pairs.

4.1 Differential Analysis of the Nonlinear Layer

The operator χ (or S-box S) is the unique nonlinear component among the five transformation steps. Specifically, after the 1600-bit Keccak state is structured as a $5 \times 5 \times 64$ three-dimensional array, the χ step operates independently on each 5-bit row. Therefore, to analyze its cryptographic strength, we investigate the differential propagation through $\chi : \mathbb{F}_2^5 \to \mathbb{F}_2^5$. For any given input difference Δ_{in} and output difference Δ_{out}, we define the set of valid

input values x as:

$$V(\Delta_{in}, \Delta_{out}) = \{x \in \mathbb{F}_2^5 \mid \chi(x \oplus \Delta_{in}) \oplus \chi(x) = \Delta_{out}\} \tag{4.1}$$

It is well-established that for the χ operator, $V(\Delta_{in}, \Delta_{out})$ forms an affine space over $\mathbb{F}_2$ [14], which can be characterized by both its cardinality and its underlying linear constraints.

4.1.1 Differential Distribution Table (DDT)

The Differential Distribution Table (DDT) [19] is constructed by examining the density of the differential transitions. The corresponding entry in the DDT, denoted as $\text{DDT}(\Delta_{in}, \Delta_{out})$, is defined by the cardinality of the solution set:

$$\text{DDT}(\Delta_{in}, \Delta_{out}) = \#V(\Delta_{in}, \Delta_{out}) \tag{4.2}$$

In our implementation, the DDT is generated by systematically iterating through all possible input differences Δ_{in}. For each fixed Δ_{in}, the sum of the counts across all possible output differences must satisfy the following property:

$$\sum_{\Delta_{out} \in \mathbb{F}_2^5} \#V(\Delta_{in}, \Delta_{out}) = 2^5 \tag{4.3}$$

Because the χ operator processes 5-bit rows, the total number of solutions for each row in the DDT is exactly $2^5 = 32$. This exhaustive derivation ensures that all valid differential transitions are captured, the details of which are provided in Appendix A.

4.1.2 Values of Difference Conditions Table (VDCT)

The Values of Difference Conditions Table (VDCT) [6] provides a more granular algebraic description of the differential transitions. Building upon the linear formulation of the χ operator introduced in early collision attacks [14], since $V(\Delta_{in}, \Delta_{out})$ is an affine space, it can be uniquely determined by a system of linear equations over $\mathbb{F}_2$:

$$C \cdot x = \eta \tag{4.4}$$

where $C \in \mathbb{F}_2^{5\times5}$ is a matrix representing the linear constraints and $\eta \in \mathbb{F}_2^5$ is a constant vector. For the specific χ layer, let $\Delta_{in} = (\delta_0, \delta_1, \delta_2, \delta_3, \delta_4)$. The relationship can be explicitly expressed in the following matrix form [18]:

$$\begin{bmatrix} 0 & \delta_2 & \delta_1 & 0 & 0 \\ 0 & 0 & \delta_3 & \delta_2 & 0 \\ 0 & 0 & 0 & \delta_4 & \delta_3 \\ \delta_4 & 0 & 0 & 0 & \delta_0 \\ \delta_1 & \delta_0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} \eta_0 \\ \eta_1 \\ \eta_2 \\ \eta_3 \\ \eta_4 \end{bmatrix} \tag{4.5}$$

where the elements of the target vector η are derived as:

$$\eta_i = \Delta_{out,i} \oplus \delta_i \oplus (\delta_{i+1} \oplus 1)\delta_{i+2} \tag{4.6}$$

The VDCT stores these pairs (C, η) for each valid differential $(\Delta_{in}, \Delta_{out})$. Complementarily, the Difference Conditions Table (DCT) [6] records the rank of the matrix C, denoted as k, which corresponds to the number of independent linear equations defining the affine space.

4.1.3 Probabilistic Linearization and Maximum Difference Density Subspace (MDST)

In cryptanalysis, whether applying standard differential or internal differential techniques, a critical bottleneck frequently arises during the trail connection phase: the connectivity problem. When we attempt to find a compatible initial message that satisfies a predefined differential trail, the algebraic system often becomes heavily overdefined. Specifically, for an active χ S-box, fixing a target transition $\Delta_{in} \to \Delta_{out}$ restricts the input to an affine subspace defined by a set of strict linear equations. According to the Differential Distribution Table (DDT) and the Values of Difference Conditions Table (VDCT), a transition with $DDT = 8$ requires 2 linear equations, $DDT = 4$ requires 3 equations, and $DDT = 2$ requires 4 equations. Imposing all these deterministic conditions across multiple active S-boxes rapidly depletes the available degrees of freedom, leading to widespread logical contradictions and rendering the system practically unsolvable.

To overcome this limitation, we introduce the concept of *Probabilistic Linearization*[5].

Instead of strictly enforcing all conditions to deterministically guarantee a valid input difference Δ_{in}, we relax the constraints by imposing only a single linear equation per active S-box. By doing so, we define a 4-dimensional affine subspace (containing $2^4 = 16$ elements) that encapsulates a mixture of both valid and invalid input differences. Let this single linear constraint on the 5-bit input $x = (x_0, x_1, x_2, x_3, x_4)$ be expressed as:

$$l_0 \cdot x_0 + l_1 \cdot x_1 + l_2 \cdot x_2 + l_3 \cdot x_3 + l_4 \cdot x_4 = q \tag{4.7}$$

where $(l_0, l_1, l_2, l_3, l_4, q) \in \mathbb{F}_2^6$. We denote the affine subspace defined by this equation as U. For brevity, this subspace is often represented in hexadecimal format as $Ker(\sum_{j=0}^{4} l_j \cdot 2^j, q)$.

Because the subspace U relies on only one equation, not all 16 elements within it are mathematically capable of propagating to the target Δ_{out}. We define the *Difference Density* of U as the proportion of its elements that are valid input differences for Δ_{out}. To optimize the attack, we systematically evaluate all possible 4-dimensional affine subspaces for a given Δ_{out} and identify the one that maximizes this density. The optimal subspaces for all possible non-zero output differences are compiled into the **Maximum Difference Density Subspace Table (MDST)**[5].

Furthermore, it is crucial to evaluate the algebraic cost of the valid solutions within a chosen subspace U. We define the *Average Transition Condition Number*, denoted as $E[k_U]$. For a given subspace U, $E[k_U]$ is calculated by extracting all *valid* input differences contained within it, retrieving their respective transition condition numbers (i.e., the rank k from the DCT), and computing the mathematical average of these ranks. This metric accurately reflects the expected equation burden a valid solution from this subspace will impose on the subsequent solving phases.

By implementing probabilistic linearization guided by the MDST, the methodology for finding the initial messages shifts from a deterministic search to a probabilistic filtering strategy. When fixing the target output difference, we no longer enforce the exact input difference immediately. Instead, we select the single optimal equation provided by the MDST to construct a highly relaxed, probabilistic input difference system. Since this relaxed system drastically reduces the total number of required equations, the degrees of freedom are preserved, yielding a significantly larger solution space of eligible initial messages. While this candidate pool inevitably contains invalid transitions, generating it requires minimal com-

putational effort. Ultimately, these candidate messages can be rapidly verified using simple lookup tables, allowing us to efficiently filter out invalid states and isolate the exact messages that conform to the target input difference.

4.2 The Target Difference Algorithm Framework

In previous literature, such as the work by Dinur et al. [14], the term Target Difference Algorithm (TDA) typically refers specifically to the algebraic phase of connecting the initial state to a differential trail. In this paper, however, we expand the scope of TDA to denote the comprehensive, end-to-end framework employed for finding collisions.

Our complete TDA framework is explicitly structured into three primary phases:

1. **Trail Discovery:** Searching for a suitable, low-weight differential trail over the target rounds.

2. **Connector Construction:** Algebraically linking the initial state to the trail by exploiting the linearization of the χ operation (S-box). Our strategy involves incrementally building these connectors: we initially construct a 2-round connector optimized by non-full linearization techniques, and subsequently extend it to form a 3-round connector.

3. **Collision Search:** Performing an efficient search over the large subspace of messages that successfully satisfy the connector's algebraic equations to find the actual colliding pair.

In the remainder of this section, we detail the methodology and implementation of each phase within this extended TDA framework.

4.2.1 Trail Discovery with SAT Solver

Before detailing the search strategy, we define the differential state notation used throughout this paper. Let α_i denote the state difference at the beginning of round i, and β_i denote the state difference after the linear layer L (comprising the θ, ρ, and π step mappings) but immediately before the non-linear χ operation. Consequently, the differential propagation through a single round is mathematically represented as $\alpha_i \xrightarrow{L} \beta_i \xrightarrow{\chi} \alpha_{i+1}$.

Core SAT Formulation

In this phase, we utilize a SAT solver to discover a low-weight differential trail over three rounds tailored for the Target Difference Algorithm (TDA). Because TDA focuses on the fundamental properties of the unkeyed permutation, the asymmetric ι (iota) constant addition is omitted from the formulation. Minimizing the trail weight ensures a higher differential probability and significantly reduces the occurrence of algebraic contradictions during the subsequent Gaussian elimination phase.

Specifically, the solver is deployed to establish the core trail spanning from α_2 to α_5. The core SAT formulation encodes the state differences and structural transitions. To strictly bound the overall trail weight and ensure the search remains computationally feasible, we apply cardinality constraints utilizing a Sequential Counter encoding. For a three-round search aiming for a maximum differential weight of $W \leq 19$, there are $n = 960$ potential active S-box variables (320 per round). The sequential encoding introduces exactly $n \times (W + 1) = 19,200$ auxiliary bit-counter variables. This precise constraint model forces the solver to isolate and return only the differential trails that satisfy the predefined weight threshold.

Instance-Specific Boundary Constraints

To ensure the solver produces a valid collision for a specific target instance, such as SHAKE128 ($r = 1344$, $c = 256$), the search space must be bounded by strict unit clauses. The core trail discovery focuses on the final three rounds, establishing the differential path from α_2 to α_5. Because SHAKE128 allows for a variable output length d, achieving a successful collision requires that no differences are observed in the digested hash output. Consequently, the first d bits of the final state difference at α_5 are explicitly constrained to zero. Furthermore, to prevent the SAT solver from returning a mathematically valid but practically useless trivial solution (an all-zero state where no differences exist), a specific clause is appended mandating that at least one bit within the unconstrained state must evaluate to true.

Subsequently, when this core trail is extended backward to deduce the initial state α_0, strict input boundary conditions must be enforced. The initial state α_0 must not contain any differences in its capacity and padding segments. Therefore, for the SHAKE128 instance, the last 256 bits of the capacity and the shortest 4-bit padding segment of α_0 (amounting

to 260 bits) are explicitly constrained to zero. This successfully isolates the initial message differences strictly within the 1344-bit rate portion.

Backward Extension to β_1 and β_0

Once α_2 is determined by the forward search, the subsequent objective is to derive the preceding differences, β_1 and β_0, to extend the trail backward. Our strategy initially involved identifying a loosely constrained β_1 via the Differential Distribution Table (DDT) to minimize potential algebraic conditions. However, although β_1 was carefully selected to mitigate the risk of local contradictions, it remains highly susceptible to global incompatibilities with subsequent differences, such as β_2. Consequently, if the automated search persistently fails to converge on a valid path to α_2, it strongly indicates an inherent flaw in the initial choice of β_1. Furthermore, subsequent verification frequently revealed that most generated β_0 candidates were algebraically inconsistent with β_1. Due to computational resource limitations in our experimental environment, discovering a strictly valid β_0 using this pure search-based approach proved infeasible. The specific methodology for testing the algebraic consistency of β_0 is detailed in the following sections, and the implementation code for searching β_1 and β_0 is provided in Appendix C.1. Because of these persistent inconsistencies, the specific β_0 configuration provided in the reference literature was ultimately adopted for the subsequent analysis.

While β_0 is anchored to be locally compatible with β_1, this does not guarantee global compatibility across the entire candidate trail from α_2 to α_5. Furthermore, the connector construction phase itself does not verify the correctness of this trail; we systematically build the connector conditioned on an assumed, potentially flawed trajectory. Consequently, any underlying algebraic contradictions that render the path impossible (i.e., resulting in a probability of zero) will only be exposed during the final searching phase.

4.2.2 Construction of the 2-Round Connector

The core idea of the 2-round connector is to convert the problem of bypassing the first two rounds into solving a system of linear equations. The first two rounds can be expressed as $\chi_1 \circ L \circ \chi_0 \circ L$.

Deriving Linear Systems E_M and E_z

To successfully bridge the initial state with the differential trail, we formulate two primary systems of linear equations. Let x and y denote the input and output states of the first non-linear layer χ_0, respectively, and let z denote the input state to the second non-linear layer χ_1.

The first system, E_M, is formulated over the variables of x. It is designed to satisfy the strict capacity and padding constraints inherent to the initial state α_0. By propagating these deterministic, predefined bits of the initial message forward through the first linear layer L, we derive the necessary linear equations that constitute E_M over x.

The second system, E_z, is constructed based on the conditions required to satisfy the differential trail. As the differential trail for the subsequent rounds has been fully established in the previous phase, the state z is confined to a specific affine subspace determined by the valid transitions from β_1 to α_2. These transition conditions can be systematically extracted using the Values of Difference Conditions Table (VDCT), allowing us to express the constraints on z as a linear system:

$$B \cdot z = t_B \tag{4.8}$$

where B is the coefficient matrix corresponding to the VDCT conditions, and t_B is the respective constant vector.

To unify our algebraic systems around the first non-linear layer χ_0, we must propagate these constraints backward from z to y. The algebraic relationship between y and z is defined by the linear layer L and the first round constant $RC[0]$:

$$L \cdot (y \oplus RC[0]) = z \implies y = (L^{-1} \cdot z) \oplus RC[0] \tag{4.9}$$

By substituting z with $L \cdot (y \oplus RC[0])$ into our VDCT constraint equation, we effectively map the restrictions on z into a system of linear constraints directly over the variables of y. The explicit derivation and computational implementation of the inverse linear transformation L^{-1} are detailed in the Appendix. This backward mapping systematically yields our second foundational system, E_z, expressed entirely in terms of the state y.

Tracking Active Bits via the Flag Vector U

To systematically identify which specific bits require linearization, we utilize the linear system E_z derived in the previous step. Recall that E_z has been algebraically transformed into a system of equations strictly over y, which represents the output state of the first non-linear layer χ_0. By projecting the coefficient matrix of these equations, we can extract a 1600-bit flag vector U that maps directly to the Keccak state.

Specifically, we partition this 1600-bit vector to align with the state architecture, defining a 5-bit flag vector

$$U_m = (u_{0,m}, u_{1,m}, u_{2,m}, u_{3,m}, u_{4,m})$$

for each 5-bit column m (where $0 \leq m \leq 319$). For any state bit $y[i][m]$, if it is actively involved as a variable within the equations of E_z, its corresponding flag is set to $u_{i,m} = 1$; otherwise, it is set to 0. Ultimately, this comprehensive flag vector U explicitly dictates the necessary extent of linearization required for each individual S-box in χ_0.

4.2.3 Optimizing Degrees of Freedom: S-box Linearization

A significant challenge in uniting E_M and E_z is the rapid depletion of degrees of freedom. A full linearization of all S-boxes in χ_0 consumes excessive degrees of freedom, potentially rendering the solution space too small to find a valid collision. Therefore, we selectively apply different linearization techniques based on the flag vector U.

Full S-box Linearization

For an S-box where $U_i = 11111_2$, all 5 output bits are involved in the subsequent algebraic constraints. Fully linearizing such an S-box strictly requires imposing at least 3 linear equations on its input bits to confine the inputs to a 2-dimensional linearizable affine subspace.

Non-full S-box Linearization

To preserve degrees of freedom, we apply non-full S-box linearization to S-boxes where $U_i \neq 11111_2$. The key observation is that uniting E_M and E_z does not necessitate the full linearization of every S-box in χ_0.

- **Non-active S-boxes:** If only a subset of the output bits requires linearization, we introduce at most 2 binary linear equations over the input bits, effectively saving at least 1 degree of freedom compared to full linearization. If $U_i = 0$, no linearization equations are added.

- **Active S-boxes with DDT entry 8:** For an active S-box with a Differential Distribution Table (DDT) entry of 8, the input is already restricted to a solution set via 2 linear equations. Under these constraints, 4 out of the 5 output bits are inherently linear. If the single non-linear bit is not flagged by U_i, zero additional equations are required to linearize the influencing outputs.

By extensively exploiting non-full linearization, the dimension of the solution space for the 2-round connector is significantly enlarged, making practical collision attacks feasible.

Algebraic Consistency Verification for β_0

Building upon the established linearization strategy, we must rigorously verify the validity of candidate intermediate differences β_0. It is crucial to emphasize that while the S-box linearization process remains an integral part of this verification, it must be applied in a phased manner. Before introducing any auxiliary linear equations—specifically the hypothetical constraints required to resolve active S-boxes with a Differential Distribution Table (DDT) entry of 8, as well as the selectively retained base equations for inactive S-boxes (DDT=32)—it is imperative to confirm that the deterministic base system inherently lacks algebraic contradictions.

To achieve this, we selectively filter the system E_z to establish a preliminary consistency test. Specifically, we isolate a deterministic subsystem from E_z comprising exclusively those equations that are unconditionally defined by the initial differential propagation and the exact, unambiguous S-box linearizations. Equations containing variables that rely on the subsequent addition of auxiliary stochastic constraints (such as those for the DDT=8 and DDT=32 configurations) are temporarily excluded from consideration. By utilizing this restricted, inherently contradiction-free subsystem, we can efficiently and accurately evaluate whether a candidate β_0 is algebraically consistent with the predefined β_1 early in the process. This rigorous pruning phase guarantees that only viable β_0 configurations proceed to the full algebraic connector construction, where the remaining auxiliary equations are ultimately integrated.

4.2.4 Extension to the Adaptive 3-Round Connector

The methodology presented in this section is primarily based on the work of Song et al. [12]. To mitigate the exceptionally low probability of the subsequent differential trail, we extend the connector to three rounds. This directly generates message subspaces that deterministically satisfy the initial 3-round differential conditions, drastically reducing the time complexity of the final brute-force search stage.

System Translation and Construction

Omitting the constant addition mapping, let x and z be the input states of χ_0 and χ_1, respectively. Suppose the linear equation system derived over the initial rounds is denoted by $E_M : A \cdot x = t_0$. By applying full linearization to χ_0, we directly translate E_M into an equivalent linear system expressed over the state variables of z:

$$E'_M : A' \cdot z = t'_0$$

Once the system is expressed over z, the remaining procedure strictly follows the standard 2-round connector methodology. Specifically, non-full linearization is applied to χ_1 to successfully unite E'_M with the algebraic constraints introduced by the third round (χ_2). To ensure the overarching system remains consistent, we simply maintain valid partial linearization configurations for any S-boxes in χ_1 that are already partially constrained by E'_M, systematically avoiding internal algebraic contradictions.

4.2.5 Collision Search and Two-Block Extension Strategy

During the collision search phase, the candidate message pool generated from a single block may occasionally be insufficient to yield a valid collision. This occurs when the available degrees of freedom within a single message block are exhausted by the strict algebraic constraints of the connectors. To overcome this limitation and dynamically expand the search space, we utilize a two-block message extension strategy [7].

Following the standard Keccak sponge construction, processing a two-block message involves a sequential absorption phase. Initially, the first message block is absorbed and pro-

cessed through the target reduced-round Keccak permutation (e.g., 5 rounds). The resulting intermediate output state is then XORed with the second message block.

This mechanism provides a critical advantage for our algebraic attack. In a single-block scenario, the capacity bits are strictly fixed to predefined initial values (typically zeros), which tightly constrains the initial algebraic system E_M. However, in a two-block scenario, the capacity portion of the state entering the second block is randomized by the output of the first block's permutation.

By varying the content of the first message block, we can generate entirely new, pseudo-random capacity values for the second block. This variation allows us to reconstruct the initial equation system E_M under different capacity conditions, effectively replenishing the consumed degrees of freedom. Consequently, this multi-block approach enables the generation of a fresh and virtually unlimited pool of candidate messages, thereby effectively bypassing the bottleneck of search space exhaustion and ensuring the practical feasibility of finding a valid collision.

4.3 The Target Internal Difference Algorithm Framework

To address the probability barriers encountered during the squeeze phase of standard standard differentials, the Target Internal Difference Algorithm (TIDA) framework leverages the translation invariance of the Keccak permutation. Instead of tracking external differences between two independent message states, this framework analyzes the internal difference propagation within a single state vector. As a comprehensive, end-to-end cryptanalytic system, the TIDA framework shifts the connection paradigm from a deterministic path search to a statistical subspace alignment, significantly improving collision probabilities during the final squeezing phase.

Overall, the complete TIDA framework is structured into three primary stages:

1. **Message Selection:** Formulating the initial algebraic systems to select starting message candidates that satisfy the baseline capacity and padding constraints.

2. **Message Collection:** Iteratively solving the internal differential transition systems to accumulate a sufficiently large pool of valid intermediate states that successfully conform to the designated trail.

3. **Collision Search:** Executing a specialized birthday search within the highly restricted target subspace during the squeeze phase to efficiently identify the final colliding message pair.

In the remainder of this section, we detail the theoretical motivations, complexity thresholds, and procedural phases that constitute this specialized internal differential framework.

4.3.1 The Squeeze Attack: A Variant of Birthday Attack

In the preceding chapters, we explored the Target Difference Algorithm (TDA), which seeks a deterministic zero-difference to produce collisions. Because TDA aims to perfectly cancel out differences before or at the beginning of the squeezing phase, it does not inherently rely on the statistical distribution of the output space. Consequently, the concept of a specialized squeeze-phase attack was not necessary for TDA.

However, when transitioning to internal differentials, achieving a perfect zero-difference becomes mathematically impossible due to the rigid translation-invariant constraints. To circumvent this, we must introduce a completely different collision search strategy that is uniquely required for the Target Internal Difference Algorithm (TIDA). In 2012, Dinur et al. [15] pioneered this strategy, termed the **Squeeze Attack**, which maps a massive set of input states into a highly restricted subset of possible outputs during the squeezing phase.

Following the mathematical framework established by Zhang et al.[6], this process is formally characterized as a **variant of the birthday attack**. The core principle is to perform a collision search strictly within a low-dimensional affine subspace, denoted as $\mathcal{V}$, which is generated by the internal differential characteristic.

In a generic birthday attack against a hash function with an n-bit digest, the expected time complexity to identify a collision is approximately $2^{n/2}$. However, the use of internal differentials significantly restricts the effective search space. By generating message candidates that satisfy the specific conditions of the internal difference, the resulting digests are forced into the subspace $\mathcal{V}$ with a high probability. Consequently, the attacker can leverage the birthday paradox within this constrained target space to find a collision with a complexity much lower than the theoretical bound of a standard attack.

This shift from the deterministic trail search of TDA to a statistical subspace search provides the fundamental theoretical motivation for TIDA. In the subsequent sections, we will

detail how to construct the necessary linear systems to efficiently generate message candidates that conform to these internal differential conditions, thereby executing the attack.

4.3.2 Subspace Mapping and Complexity Analysis

Let H be a hash function with an output space of size 2^n. According to the standard birthday paradox, the probability that no collision is found after 2^t randomly selected inputs is approximated by:

$$1 - P \approx \prod_{i=1}^{2^t-1} \left(1 - \frac{i}{2^n}\right) \approx e^{-\frac{2^t(2^t-1)}{2^{n+1}}}$$

where P denotes the probability of finding at least one collision.

We now introduce a variant of the birthday attack by partitioning the input domain into 2^k disjoint subsets, denoted as $S_1, S_2, \ldots, S_{2^k}$, which map to corresponding output subsets $D_1, D_2, \ldots, D_{2^k}$. Let the cardinality of each input subset be $|S_j| = 2^l$ and the cardinality of each output subset be $|D_j| = 2^m$ for all $j \in \{1, 2, \ldots, 2^k\}$. Assuming that we search through 2^w (where $w \leq k$) such subsets to find a collision, the overall collision probability P can be approximated as:

$$1 - P \approx \left(e^{-\frac{2^l(2^l-1)}{2^{m+1}}}\right)^{2^w} = e^{-\frac{2^{l+w}(2^l-1)}{2^{m+1}}}$$

Consequently, we must determine the total number of required inputs, N, to guarantee a high probability of finding a collision. The overall complexity N is dictated by two distinct bottlenecks: the requirement to populate the subsets and the execution of the birthday search itself.

First, for a collision to even be possible within a specific subset, that subset must contain at least two elements on average. Since an input randomly falls into a specific subset with a probability of 2^{-k}, satisfying this condition requires $2 \times 2^k = 2^{k+1}$ inputs. Second, once the subsets are sufficiently populated, finding a collision across 2^w subsets of size 2^m follows the standard birthday bound, requiring $2^{(m+w)/2}$ inputs.

To rigorously understand why $k = m$ serves as the critical watershed between these two bottlenecks, consider the extreme case where we exhaustively search all available subsets ($w = k$). In this scenario, the theoretical birthday bound suggests a required input size of $N = 2^{(m+k)/2}$. Since these N inputs are uniformly distributed across the 2^k subsets, the expected number of elements allocated to each subset is $N/2^k = 2^{(m-k)/2}$.

If $k = m$, this expected value reduces to $2^0 = 1$. If $k > m$, the expectation drops below 1. In both cases, relying on the theoretical birthday bound leads to a logical contradiction, as it is mathematically impossible to form a collision pair within a subset containing fewer than two elements. Therefore, when $k \geq m$, the birthday bound significantly underestimates the actual requirement, and the overall complexity is strictly dominated by the data collection phase (i.e., we must forcefully collect $N = 2^{k+1}$ inputs just to guarantee the existence of pairs). Conversely, when $k < m$, the subsets naturally accumulate more than enough elements, making the birthday search ($2^{(m+w)/2}$) the true bottleneck. Thus, N is formally defined as:

$$N = \begin{cases} 2^{(m+w)/2}, & \text{if } k < m \\ 2^{k+1}, & \text{if } k \geq m \end{cases}$$

4.3.3 Theoretical Framework: Internal Symmetry and Overlapping Constraints

The feasibility of internal differential cryptanalysis on Keccak relies on the specific algebraic properties of its round functions [15]. The four primary transformations—θ, ρ, π, and χ—exhibit strict translation invariance along the z-axis. Mathematically, this symmetry implies that applying these operators to the entire 1,600-bit state is equivalent to processing independent or coupled intervals along the z-axis separately. This periodic structure allows for the construction of internal differential trails by analyzing the state's internal dependencies. Although the ι step introduces asymmetry through the addition of round constants, its impact on the overall differential trail is relatively localized due to the low Hamming weight of these constants, which ensures that the internal symmetry remains approximately preserved for most bit positions.

By exploiting this symmetry property, the 64-bit lanes can be partitioned into smaller, periodic segments. For instance, dividing the lanes into 8 distinct parts reduces the number of independent state variables from 1,600 to 200. Similarly, partitioning into 2 parts reduces the variables to 800, a configuration frequently utilized in existing literature for internal differential attacks [6][5]. While this dimensionality reduction significantly narrows the search space, the number of derived algebraic equations does not decrease proportionally.

Consequently, the resulting linear system becomes heavily overdefined, making it highly susceptible to logical contradictions during the solving phase. To mitigate this issue and reduce the rate of contradiction, we incorporate the probabilistic linearization technique [5]. By relaxing the deterministic constraints on the non-linear layer, probabilistic linearization effectively reduces the total number of required equations, thereby ensuring the consistency of the algebraic system and making the internal connectivity problem computationally feasible.

To formally illustrate this reduced state representation, consider partitioning each 64-bit lane into 8 equal segments (corresponding to a translation period of 8). The state at the t-th stage can be expressed as:

$$A^{(t)} = \left(A_0^{(t)}, A_1^{(t)}, A_2^{(t)}, A_3^{(t)}, A_4^{(t)}, A_5^{(t)}, A_6^{(t)}, A_7^{(t)}\right)$$

Since these individual segments do not possess intrinsic symmetry themselves, we construct the internal differential trail by establishing a relative reference. Specifically, we designate one segment—typically the last one, $A_7^{(t)}$—as the base state and normalize it to 0. Consequently, the remaining segments represent the internal differences relative to this base state. Furthermore, because the asymmetric round constant (e.g., $RC[3]$) introduced by the ι operation is a linear addition, it can be directly absorbed into the internal difference of the affected segment. Therefore, the combined effect of the χ and ι operations can be cleanly represented as:

$$\iota \circ \chi(A^{(2.5)}) = \chi(A^{(2.5)}) \oplus RC[3] = A^{(3)}$$

By fusing the round constant into the internal difference representation, the algebraic transitions across the non-linear layer are effectively decoupled from the absolute state values. This algebraic simplification isolates the differential transitions, allowing us to evaluate the trail using the generalized relation $A_i^{(t)} = \chi(A_i^{(t-0.5)})$ without being constrained by the absolute values of the base state.

Unlike standard differential cryptanalysis, where bit transitions are typically treated as independent events, internal differentials derived from the self-translation $A \oplus \tau(A)$ introduce a complex phenomenon of overlapping constraints. When deriving the algebraic conditions through the Value of Difference Conditions Table (VDCT), a constraint imposed on a specific bit $A[x, y, z]$ may simultaneously restrict its translated counterparts due to the inherent

periodicity along the z-axis. Consequently, the transition probability of an internal difference cannot be calculated as a simple product of individual bit probabilities, as is common in TDA. Instead, one must rigorously evaluate the compatibility of these coupled bit conditions within the non-linear χ layer. This bit dependency necessitates a more sophisticated mathematical framework to ensure the consistency of the underlying linear system and to identify valid trails where overlapping constraints do not lead to logical contradictions.

4.3.4 Message Selection and Free Variable Extraction

The initial stage of our framework is conceptually analogous to the connector construction phase in standard TDA. Both processes share the fundamental objective: to identify initial message blocks that simultaneously satisfy the preparatory differential trail, specific domain padding rules, and capacity constraints. Therefore, this section focuses primarily on the methodological differences between the two approaches. The corresponding automated search and verification implementations are detailed in Appendix C.

In standard TDA, constructing a connector to bypass the first two or three rounds is an intricate algebraic process. Because standard differential attacks must strictly enforce external message formatting while coordinating constraints across multiple active rows, the attacker must systematically solve a heavily constrained linear system to identify valid bit assignments within the rate portion.

However, the execution of message selection within our TIDA framework is significantly more straightforward due to the exceptionally low propagation weight of our target internal differential trail across the initial rounds. Consequently, the vast majority of the χ S-boxes in this phase remain inactive, maintaining zero input and output internal differences ($\Delta_{in} = \Delta_{out} = 0$). Because the non-linear layer imposes minimal and highly sparse algebraic constraints during these early rounds, the initial system does not heavily deplete the available state space.

To systematically extract the exact bit positions available for randomized initial assignment, we execute an automated backward-propagation procedure. First, we derive the linear constraint equations corresponding to the active S-boxes in the second round. Using the inverse linear layer transformation (L^{-1}), these second-round constraints are propagated backward to the initial state matrix. Next, we integrate these back-propagated equations with the

first-round S-box constraints, simultaneously substituting the fixed bit values dictated by the standard padding rules and constant capacity parameters.

Upon applying Gaussian elimination to this unified linear system, we reduce the equations to their simplified row echelon form. This allows us to systematically evaluate the remaining unconstrained positions in the state array. Specifically, the bit indices within the rate portion that are not strictly determined by the differential constraints—meaning they do not serve as pivot variables—are completely independent. In our framework, we formally define these fully independent and assignable bit positions as **free variables**. Because the extensive structural sparsity of the initial rounds preserves an ample amount of these free variables, we can directly assign random values to them. Through this randomized assignment, we highly efficiently generate a robust candidate pool of valid starting messages that perfectly conform to the preliminary trail conditions, bypassing the high computational overhead typically associated with complex multi-round equation solvers.

4.3.5 Message Collection and Subspace Collision Search

Once the initial message selection phase is complete, we obtain a large pool of candidate messages that successfully conform to the 2.5-round internal differential trail. We theoretically guarantee that all these filtered messages successfully propagate through the non-linear χ operation of the third round and reach the categorization stage. Due to the algebraic properties of χ, a single input internal difference can transition into multiple possible output internal differences. To systematically optimize our approach, we categorize these candidate messages into disjoint sets based on their specific input and output internal difference pairs across this χ mapping.

The fundamental rationale behind this categorization strategy relies on the strict algebraic conditions required for a final collision. If two messages produce a collision at the fourth round, they must share the exact same values in the target bits. Given the deterministic transition $A_i^{(t)} = \chi(A_i^{(t-0.5)})$ within the same affine subspace, we can mathematically deduce that if two messages collide at the output, their intermediate states at the preceding half-round $(t-0.5)$ must also be perfectly identical. This deterministic traceback allows us to accurately guarantee the equivalence of their prior states.

During the searching stage, we evaluate the candidate messages globally across all cate-

gorized sets. Based on the aforementioned trace-back conditions, if two messages fall into different sets, they possess distinct output internal differences after the third round. This divergence mathematically guarantees that their subsequent states will follow different differential trails, making a final collision practically impossible.

Conversely, messages grouped within the same set share the exact same internal difference configuration. This precise alignment perfectly fulfills the prior state equivalence, confining the states to a strictly controlled affine subspace where the differential conditions are matched, making a collision almost guaranteed. Since the candidate messages are uniformly distributed among the categorized sets, the overall probability of generating a full collision is elegantly transformed into a direct function of the total number of disjoint sets.

4.4 Comparative Analysis of TDA and TIDA

To thoroughly understand the characteristics of the Target Difference Algorithm (TDA) and the Target Internal Difference Algorithm (TIDA), it is essential to evaluate their respective theoretical frameworks and practical trade-offs objectively. It is important to note that TIDA is not a universal replacement for TDA; rather, both algorithms exploit different structural properties of Keccak and are suited for different cryptanalytic scenarios.

The Target Difference Algorithm (TDA)

TDA processes standard differentials, focusing on the difference between two distinct and independent message states, denoted as $\Delta = A \oplus B$.

- **Advantages:** The primary strength of TDA lies in its generality and the independence of its state variables. As a fundamental differential approach, it is a universally applicable technique successfully deployed against a wide variety of cryptographic primitives. In the context of Keccak, TDA holds a distinct structural advantage: the linear addition of the round constants (ι step) perfectly cancels out in the state difference, entirely removing it from the cryptanalytic modeling. Furthermore, its intra-round constraints are inherently consistent. As long as the bit transitions comply with the standard Difference Distribution Table (DDT), logical contradictions do not naturally occur within a single round. This freedom allows the attacker significant flexibility in selecting trails,

which is why standard TDA currently holds the records for penetrating the highest number of rounds in most Keccak variants.

- **Limitations:** The critical bottleneck of TDA is the exponential growth of the search space. Although contradictions are typically absent within a single round, they inevitably arise and accumulate as the algebraic system spans across multiple rounds. While techniques like the Maximum Difference Density Subspace (MDST) can be employed to manage these cross-round contradictions, the probability of finding a specific target difference in a standard squeeze attack still drops significantly. Consequently, constructing a full collision often requires solving an overwhelmingly large linear system, making the squeeze phase highly inefficient or computationally prohibitive for certain high-security variants.

The Target Internal Difference Algorithm (TIDA)

To address the probability barriers during the squeeze phase, TIDA leverages the translation invariance of the Keccak permutation, analyzing the internal difference within a single state, defined as $\Delta = A \oplus \tau(A)$.

- **Advantages:** By exploiting internal symmetries, TIDA drastically reduces the target output space. This mapping of a massive number of inputs to a restricted subset of outputs yields a significantly higher collision probability during the squeeze phase compared to standard differentials. Combined with probabilistic linearization, TIDA can effectively reduce the dimension of the inner system, providing a highly efficient attack vector for specific reduced-round scenarios.

- **Limitations:** The restrictive nature of TIDA is its most significant drawback. Because internal differentials rely on self-translation, applying the Difference Conditions Table (DCT) introduces severe constraint coupling. This coupling frequently causes overlapping constraints to generate logical contradictions **immediately within a single round** (e.g., restrictions on bit $A[x, y, z]$ invariably affect its translated counterpart). Managing and resolving these immediate intra-round inconsistencies significantly complicates the cryptanalytic process. Furthermore, TIDA is highly specialized; its absolute reliance on strict translation invariance makes it difficult to generalize to other ciphers

lacking such symmetry, effectively limiting its trail-selection flexibility compared to TDA.

Summary

In conclusion, TDA remains the superior choice for maximizing the absolute number of attacked rounds due to its generality, ι-cancellation, and lack of intra-round structural restrictions. Conversely, TIDA provides a highly specialized mathematical framework that dramatically improves collision probabilities within the squeeze phase at the cost of strict symmetry constraints and complex, intra-round constraint coupling. The exploration of TIDA in this chapter serves to demonstrate how specific internal structural symmetries can be exploited to optimize the inner system size in reduced-round Keccak collisions.

References

[1] National Institute of Standards and Technology, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, Federal Information Processing Standards Publication (FIPS PUB) 202, U.S. Department of Commerce, 2015.

[2] T. Li and Y. Sun, "Preimage attacks on round-reduced Keccak-224/256 via an allocating approach," in *Advances in Cryptology – EUROCRYPT 2019*, pp. 556–584, 2019.

[3] Z. Zhang, C. Hou, and M. Liu, "Preimage attacks on up to 5 rounds of SHA-3 using internal differentials," in *Advances in Cryptology – EUROCRYPT 2025*, pp. 333–363, 2025.

[4] T. Li, Y. Sun, M. Liao, and D. Wang, "Preimage attacks on the round-reduced Keccak with cross-linear structures," *IACR Transactions on Symmetric Cryptology*, vol. 2017, no. 4, pp. 39–57, 2017.

[5] Z. Zhang, C. Hou, and M. Liu, "Probabilistic Linearization: Internal Differential Collisions in up to 6 Rounds of SHA-3," *Cryptology ePrint Archive*, Report 2024/1136, 2024.

[6] Z. Zhang, C. Hou, and M. Liu, "Collision Attacks on Round-Reduced SHA-3 Using Conditional Internal Differentials," in *EUROCRYPT 2023*, LNCS 14007, pp. 220–251, 2023.

[7] S. Huang, O. A. Ben-Yehuda, O. Dunkelman, and A. Maximov, "Finding Collisions against 4-Round SHA-3-384 in Practical Time," *IACR Trans. Symmetric Cryptol.*, vol. 2022, no. 3, pp. 239–270, 2022.

[8] J. Guo, G. Liao, G. Liu, M. Liu, K. Qiao, and L. Song, "Practical Collision Attacks against Round-Reduced SHA-3," *Journal of Cryptology*, vol. 33, no. 1, pp. 148–185, 2020.

[9] F. de Lima Marquezino, R. Portugal, and C. Lavor, *A Primer on Quantum Computing*, SpringerBriefs in Computer Science, Springer, 2019.

[10] K. Qiao, L. Song, M. Liu, and J. Guo, "New Collision Attacks on Round-Reduced Keccak," in *EUROCRYPT 2017*, LNCS 10211, pp. 216–243, 2017.

[11] S. Huang, X. Wang, G. Xu, M. Wang, and J. Zhao, "Conditional Cube Attack on Reduced-Round Keccak Sponge Function," in *EUROCRYPT 2017*, LNCS 10211, pp. 492–521, 2017.

[12] L. Song, G. Liao, and J. Guo, "Non-full Sbox Linearization: Applications to Collision Attacks on Round-Reduced KECCAK," in *CRYPTO 2017*, LNCS 10402, pp. 428–451, 2017.

[13] S. Moriai (Ed.), *Fast Software Encryption - FSE 2013*, LNCS 8424, Springer, 2014.

[14] I. Dinur, O. Dunkelman, and A. Shamir, "New Attacks on Keccak-224 and Keccak-256," in *FSE 2012*, LNCS 7549, pp. 442–461, 2012.

[15] I. Dinur, O. Dunkelman, and A. Shamir, "Collision Attacks on Up to 5 Rounds of SHA-3 Using Generalized Internal Differentials," *Cryptology ePrint Archive*, Report 2012/672, 2012.

[16] M. Soos, K. Nohl, and C. Castelluccia, "Extending SAT Solvers to Cryptographic Problems," in *SAT 2009*, LNCS 5584, pp. 244–257, 2009.

[17] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," in *CP 2005*, LNCS 3709, pp. 827–831, 2005.

[18] J. Guo, M. Liu, and L. Song, "Linear Structures: Applications to Cryptanalysis of Round-Reduced KECCAK," in *ASIACRYPT 2016*, LNCS 10031, pp. 249–274, 2016.

[19] J. Daemen and G. Van Assche, "Differential Propagation Analysis of Keccak," in *FSE 2012*, LNCS 7549, pp. 422–441, 2012.

[20] A. Duc, J. Guo, T. Peyrin, and L. Wei, “Unaligned Rebound Attack: Application to Keccak,” in *FSE 2012*, LNCS 7549, pp. 407–426, 2012.

[21] S. Mella, J. Daemen, and G. Van Assche, “New techniques for trail bounds and application to differential trails in Keccak,” *IACR Transactions on Symmetric Cryptology*, vol. 2017, no. 1, pp. 329–357, 2017.

[22] G. Liu, W. Qiu, and Y. Tu, “New Techniques for Searching Differential Trails in Keccak,” in *IACR Transactions on Symmetric Cryptology*, Vol. 2019, No. 4, pp. 407–437, 2020.

Appendix A

Inverse of the Linear Layer

In this chapter, we derive the inverse operations for the linear steps of the Keccak permutation, specifically θ, ρ, and π. Among these, computing the inverse of the θ step presents the greatest complexity.We represent the θ, ρ, π transformation as a 1600×1600 matrix over $\mathbb{F}_2$ and subsequently compute its inverse using Gaussian elimination.

Listing A.1: Inverse of Linear Layer

```
#include <iostream>
#include <vector>
#include <stdint.h>

using namespace std;

typedef uint64_t u64;
typedef vector<vector<int>> Matrix2D;

// ==========================================
// 1. Initial Input Data and Constants
// ==========================================
const u64 delta_in1[25] = {
    0x0000000000000001, 0x0000000000000000, 0x0000000000000008, 0x0000000000000000, 0x0000000000000000,
    0x0000000000000008, 0x0000000000000000, 0x0000000000000008, 0x0000000000000000, 0x0000000000000000,
    0x0000000000000000, 0x0000000040000000, 0x0000000000000000, 0x0000000000000000, 0x0000000000000000,
    0x0000000000000000, 0x0000000000000000, 0x0000000000000000, 0x0000000000000000, 0x0000000000000000,
    0x0000000000000000, 0x0000000040000000, 0x0000000000000000, 0x0000000000000000, 0x0000000000000000
};

const int r_ro[5][5] = {
    { 0, 36,  3, 41, 18},
    { 1, 44, 10, 45,  2},
    {62,  6, 43, 15, 61},
    {28, 55, 25, 21, 56},
    {27, 20, 39,  8, 14}
};

// ==========================================
// 2. Target Vector Processing (Pure Flattening)
// ==========================================
void buildStateMatrix(const u64 in[25], int B[5][5][64]) {
    u64 A_1[5][5];
```

```
    for(int j = 0; j < 25; j++) A_1[j%5][j/5] = in[j];
    for(int j = 0; j < 25; j++){
        u64 x = 1;
        for(int i = 0; i < 64; i++ ){
            B[j%5][j/5][i] = (A_1[j%5][j/5]/x)%2;
            x *= 2;
        }
    }
}

vector<int> getTargetVector() {
    int B[5][5][64] = {0};
    buildStateMatrix(delta_in1, B);
    vector<int> result(1600);
    for(int y = 0; y < 5; y++) {
        for(int x = 0; x < 5; x++) {
            for(int z = 0; z < 64; z++) {
                result[320 * y + 64 * x + z] = B[x][y][z];
            }
        }
    }
    return result;
}

// ==========================================
// 3. Standard FIPS 202 Forward Operations (Used to build the matrix)
// ==========================================
void theta(int in[5][5][64], int out[5][5][64]) {
    int C[5][64] = {0}, D[5][64] = {0};
    for(int x=0; x<5; x++)
        for(int z=0; z<64; z++)
            C[x][z] = in[x][0][z] ^ in[x][1][z] ^ in[x][2][z] ^ in[x][3][z] ^ in[x][4][z];

    for(int x=0; x<5; x++)
        for(int z=0; z<64; z++)
            D[x][z] = C[(x+4)%5][z] ^ C[(x+1)%5][(z+63)%64];

    for(int x=0; x<5; x++)
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++)
                out[x][y][z] = in[x][y][z] ^ D[x][z];
}

void rho(int in[5][5][64], int out[5][5][64]) {
    for(int x=0; x<5; x++)
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++) {
                int r = r_ro[x][y];
                out[x][y][z] = in[x][y][(z-r+64)%64];
            }
}

void pi(int in[5][5][64], int out[5][5][64]) {
    for(int x=0; x<5; x++)
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++) {
                out[y][(2*x + 3*y)%5][z] = in[x][y][z];
            }
}

// ==========================================
// 4. Construct the Complete L Matrix and Compute its Inverse
// ==========================================
Matrix2D getFullLInverseMatrix() {
```

```
    Matrix2D L_matrix(1600, vector<int>(1600, 0));
    Matrix2D C_aug(1600, vector<int>(3200, 0));
    Matrix2D inv_L(1600, vector<int>(1600, 0));

    // Step A: Generate the 1600x1600 L Matrix (Theta -> Rho -> Pi)
    for(int i = 0; i < 1600; i++) {
        int S0[5][5][64] = {0}, S1[5][5][64] = {0}, S2[5][5][64] = {0}, S3[5][5][64] = {0};

        // Set the i-th bit to 1
        int y_in = i / 320, x_in = (i / 64) % 5, z_in = i % 64;
        S0[x_in][y_in][z_in] = 1;

        // Pass through the standard linear layer
        theta(S0, S1);
        rho(S1, S2);
        pi(S2, S3);

        // Flatten the 3D output array into the i-th Column of the matrix
        for(int y = 0; y < 5; y++) {
            for(int x = 0; x < 5; x++) {
                for(int z = 0; z < 64; z++) {
                    int out_idx = 320 * y + 64 * x + z;
                    L_matrix[out_idx][i] = S3[x][y][z];
                }
            }
        }
    }

    // Step B: Set up the augmented matrix [L | I]
    for (int i = 0; i < 1600; i++) {
        for (int j = 0; j < 1600; j++) C_aug[i][j] = L_matrix[i][j];
        C_aug[i][i + 1600] = 1;
    }

    // Step C: Gaussian elimination to find the inverse matrix
    for(int i = 0; i < 1600; i++) {
        int pivot = i;
        while (pivot < 1600 && C_aug[pivot][i] == 0) pivot++;
        if (pivot == 1600) { cerr << "Matrix is not invertible!" << endl; return inv_L; }
        if (pivot != i) swap(C_aug[i], C_aug[pivot]);
        for(int r = 0; r < 1600; r++) {
            if(r != i && C_aug[r][i] == 1) {
                for(int c = i; c < 3200; c++) C_aug[r][c] ^= C_aug[i][c];
            }
        }
    }

    // Step D: Extract the right half (the inverse matrix)
    for (int i = 0; i < 1600; i++) {
        for (int j = 0; j < 1600; j++) inv_L[i][j] = C_aug[i][j + 1600];
    }
    return inv_L;
}

// ==========================================
// 5. Matrix Multiplication and Main Program
// ==========================================
vector<int> multiplyMatrixVectorGF2(const Matrix2D& matrix, const vector<int>& vec) {
    int rows = matrix.size();
    int cols = matrix[0].size();
    vector<int> result(rows, 0);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result[i] ^= (matrix[i][j] & vec[j]);
```

```
        }
    }
    return result;
}

int main() {
    // 1. Get the purely flattened target vector from delta_in1
    vector<int> targetVector = getTargetVector();

    // 2. Get the complete inverse L matrix (including Theta, Rho, Pi)
    Matrix2D fullInverseMatrix = getFullLInverseMatrix();

    // 3. Matrix-vector multiplication
    vector<int> finalResult = multiplyMatrixVectorGF2(fullInverseMatrix, targetVector);

    // 4. Output the 1600 results
    for (int i = 0; i < 1600; i++) {
        cout << finalResult[i];
        if (i < 1599) cout << ", ";
    }
    cout << endl;

    return 0;
}
```

A.1 Constructing E_M

To generate the system of equations for E_M, we modify the implementation of the inverse linear layer. In this section, we take SHAKE128 as a concrete example to demonstrate the methodology. Specifically, we replace the core logic within the `main` function with the following C++ code snippet. This correctly formulates the constraints by explicitly mapping the capacity and padding bits corresponding to the SHAKE128 parameters.

Listing A.2: Code snippet for constructing E_M (SHAKE128 example)

```
int main() {
    ofstream outFile("E_M.txt");

    if (!outFile) {
        cerr << "Error: Unable to create file!\n";
        return 1;
    }

    Matrix2D fullInverseMatrix = getFullLInverseMatrix();

    // For SHAKE128: Set the equations for the capacity bits (last c = 256 bits) to 0
    for(int i = 1344; i < 1600; i++) {
        fullInverseMatrix[i][1600] = 0;
    }

    // For SHAKE128: Set the equations for the padding bits to 1
    for(int i = 1338; i < 1344; i++) {
        fullInverseMatrix[i][1600] = 1;
    }

```

```
    // Output the extracted equations to the file
    for(int i = 1338; i < 1600; i++) {
        for(int j = 0; j < 1601; j++) {
            outFile << fullInverseMatrix[i][j] << " ";
        }
        outFile << "1\n";
    }

    return 0;
}
```

Appendix B

DDT Analysis via SAT Solving

B.1 Keccak S-box SAT Solver Modeling

In this section, we provide the C++ implementation used to generate the Conjunctive Normal Form (CNF) constraints for the Keccak S-box (χ function) differential distribution. This model allows a SAT solver to find valid differential paths.

B.2 DDT Generation for Keccak χ S-box

To facilitate the cryptanalysis of the Keccak permutation, we implement a generator for the Differential Distribution Table (DDT) of the 5-bit χ (Chi) S-box. The DDT is a fundamental tool for analyzing differential properties, tracking the frequency of each possible output difference for every given input difference.

By exhaustively iterating through all $2^5 = 32$ possible inputs, we construct a 32×32 matrix. Each entry (i, j) in the matrix represents the number of occurrences where an input difference i propagates to an output difference j. This straightforward mapping allows us to directly identify valid differential transitions and their corresponding probabilities for subsequent evaluations.

Listing B.1: Standard DDT Generation for Keccak χ S-box

```
1 #include <iostream>
2 #include <vector>
3 #include <cstdint>
4
5 using namespace std;
```

```cpp
using Matrix = vector<vector<int>>;

/**
 * Keccak Chi transformation (5-bit S-box).
 * This is the primary non-linear component
 * of the Keccak permutation.
 */
uint32_t chi(uint32_t in) {
    uint32_t out = 0;
    uint32_t x[5];

    // Extract individual bits
    for(int i=0; i<5; i++) x[i] = (in >> i) & 1;

    // Apply the algebraic formula:
    // y_i = x_i XOR ((NOT x_{i+1}) AND x_{i+2})
    for(int i=0; i<5; i++) {
        uint32_t bit = x[i] ^ ((x[(i+1)%5] ^ 1) & x[(i+2)%5]);
        out |= (bit << i);
    }
    return out;
}

/**
 * Calculates the standard Differential Distribution Table (DDT).
 * The matrix M[i][j] stores the number of times an input
 * difference 'i' leads to an output difference 'j'.
 */
Matrix calculateDDT() {
    Matrix M(32, vector<int>(32, 0));

    // 'i' is the input difference (din), 'j' is the output difference (dout)
    for(uint32_t i = 0; i < 32; i++){
        for(uint32_t j = 0; j < 32; j++){
            int n = 0;

            // Iterate over all possible 5-bit inputs x
            for(uint32_t x = 0; x < 32; x++) {
                uint32_t x_prime = x ^ i;

                // Check if the actual output difference matches 'j'
                if ((chi(x) ^ chi(x_prime)) == j) {
                    n++;
                }
            }

            // Store the frequency directly at the corresponding indices
            M[i][j] = n;
        }
    }
    return M;
}

int main() {
    // Calculate the DDT once and store the result
    Matrix ddtResult = calculateDDT();

    // Output the matrix in a standard comma-separated format
    for (int i = 0; i < 32; i++) {
        for (int j = 0; j < 32; j++) {
            cout << ddtResult[i][j] << (j == 31 ? "" : ",");
        }
        cout << endl;
    }
```

```
70
71      return 0;
72  }
```

Listing B.2: CNF Generator for Keccak S-box

```
1   #include <iostream>
2   #include <vector>
3   #include <string>
4   #include <fstream>
5   #include <cmath>
6
7   using namespace std;
8
9   // Global variables for DIMACS CNF format
10  int var_counter = 0;
11  vector<vector<int>> clauses;
12
13  // Generate a new unique variable index for the SAT solver
14  int newVar() {
15      return ++var_counter;
16  }
17
18  // Add a new clause to the formula
19  // Positive integers represent literals, negative represent their negation
20  void add_clause(const vector<int>& lits) {
21      clauses.push_back(lits);
22  }
23
24  // Implementation of the Keccak Chi (nonlinear) function: y = Chi(x)
25  uint32_t chi(uint32_t in) {
26      uint32_t out = 0;
27      uint32_t x[5];
28      for(int i=0; i<5; i++) x[i] = (in >> i) & 1;
29      for(int i=0; i<5; i++) {
30          // Chi function logic: y_i = x_i ^ ((NOT x_{i+1}) AND x_{i+2})
31          uint32_t bit = x[i] ^ ((x[(i+1)%5] ^ 1) & x[(i+2)%5]);
32          out |= (bit << i);
33      }
34      return out;
35  }
36
37  // Check if the differential transition (din -> dout) is mathematically possible
38  bool is_valid_differential(uint32_t din, uint32_t dout) {
39      // Brute-force all 32 possible 5-bit input values
40      for(uint32_t x = 0; x < 32; x++) {
41          uint32_t x_prime = x ^ din;
42          uint32_t y = chi(x);
43          uint32_t y_prime = chi(x_prime);
44          // If at least one input x satisfies the differential, it is valid
45          if ((y ^ y_prime) == dout) return true;
46      }
47      return false;
48  }
49
50  // Generate CNF constraints by excluding all impossible differential transitions
51  void add_sbox_constraints(const vector<int>& din_vars, const vector<int>& dout_vars) {
52      for(uint32_t d_in_val = 0; d_in_val < 32; d_in_val++) {
53          for(uint32_t d_out_val = 0; d_out_val < 32; d_out_val++) {
54              if (!is_valid_differential(d_in_val, d_out_val)) {
55                  // Add a forbidden clause to block this impossible transition
56                  vector<int> clause;
57                  for(int i=0; i<5; i++) {
```

```
                    bool bit_is_1 = (d_in_val >> i) & 1;
                    if (bit_is_1) clause.push_back(-din_vars[i]);
                    else          clause.push_back(din_vars[i]);
                }
                for(int i=0; i<5; i++) {
                    bool bit_is_1 = (d_out_val >> i) & 1;
                    if (bit_is_1) clause.push_back(-dout_vars[i]);
                    else          clause.push_back(dout_vars[i]);
                }
                add_clause(clause);
            }
        }
    }
}

int main() {
    vector<int> din_vars;
    vector<int> dout_vars;

    // Define 5-bit input differential variables (IDs 1-5)
    for(int i=0; i<5; i++) din_vars.push_back(newVar());
    // Define 5-bit output differential variables (IDs 6-10)
    for(int i=0; i<5; i++) dout_vars.push_back(newVar());

    // Build the Sbox differential distribution table constraints
    add_sbox_constraints(din_vars, dout_vars);

    // --- Fix Input Differential (Example: target_val = 24 / Binary 11000) ---
    int target_val = 24;
    cout << "Locking input differential to: " << target_val << " (Binary 11000)" << endl;

    for(int i=0; i<5; i++) {
        bool bit_is_1 = (target_val >> i) & 1;
        int var_id = din_vars[i];
        vector<int> unit_clause;
        // Use unit clauses to fix specific bit values for the differential
        if (bit_is_1) unit_clause.push_back(var_id);
        else          unit_clause.push_back(-var_id);
        add_clause(unit_clause);
    }

    // --- Non-Zero Constraint: Exclude the trivial all-zero differential ---
    vector<int> non_zero_clause;
    for(int var_id : din_vars) non_zero_clause.push_back(var_id);
    add_clause(non_zero_clause);

    // Output the logic to a DIMACS format file for the SAT Solver
    ofstream outfile("attack.cnf");
    outfile << "p cnf " << var_counter << " " << clauses.size() << endl;
    for(const auto& cls : clauses) {
        for(int lit : cls) outfile << lit << " ";
        outfile << "0" << endl;
    }
    outfile.close();

    cout << "CNF file 'attack.cnf' generated successfully." << endl;
    cout << "Total Variables: " << var_counter << endl;
    cout << "Total Clauses: " << clauses.size() << endl;

    return 0;
}
```

B.3 Logical Representation of Differential Constraints

To facilitate the search for differential characteristics, the constraints for the non-zero entries in the Differential Distribution Table (DDT) are stored in a Conjunctive Normal Form (CNF) file, typically named `attack.cnf`. This file can be viewed using a standard text editor, where each line represents a specific logical clause that must be satisfied.

B.3.1 Analysis of the `attack.cnf` File

A snippet of the generated `attack.cnf` file is shown below. Each numerical value represents a literal, and each line (terminated by a zero) represents a disjunctive clause.

Listing B.3: Example of constraints in attack.cnf

```
1  2  3  4  5  −6  7  8  9  10  0
−1  −2  −3  −4  −5  6  −7  −8  −9  −10  0
```

As illustrated in Listing B.3, the integers correspond to the bit positions of the input and output differences:

- **Input Difference (Δ_{in}):** Represented by variables x_1 through x_5.
- **Output Difference (Δ_{out}):** Represented by variables x_6 through x_{10}.

Logical Interpretation

The first line in the file, `1 2 3 4 5 -6 7 8 9 10 0`, defines the condition that at least one of these literals must be true. Specifically, the notation translates to the following Boolean expression:

$$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee \neg x_6 \vee x_7 \vee x_8 \vee x_9 \vee x_{10} = \text{True} \tag{B.1}$$

In terms of invalid differential transitions, this clause implies that the specific bit pattern where $(x_1, x_2, x_3, x_4, x_5, x_7, x_8, x_9, x_{10}) = 0$ and $x_6 = 1$ is impossible and is therefore excluded from the search space. Conversely, if a line were written as `-1 -2 -3 -4 -5 6 -7 -8 -9 -10 0`, it would exclude the case where all Δ_{in} bits are 1 and x_6 is 0.

By aggregating thousands of such clauses within `attack.cnf`, we effectively model the entire DDT, ensuring that the SAT solver only identifies valid differential paths where the probability is non-zero. To efficiently represent the conditions under which the Differential Distribution Table (DDT) contains non-zero entries, the transition properties are encoded into a set of logical clauses. These clauses are typically formatted for compatibility with Boolean Satisfiability (SAT) solvers, where each line represents a constraint on the input and output differences.

B.3.2 Encoding Format and Notation

Each condition generated by the algorithm consists of a sequence of integers terminated by a zero. For example, consider the following clause:

$$1 \quad 2 \quad 3 \quad 4 \quad 5 \quad -6 \quad 7 \quad 8 \quad 9 \quad 10 \quad 0 \tag{B.2}$$

The interpretation of this notation is defined as follows:

- **Variable Mapping:** The integers 1 through 5 correspond to the bit positions of the input difference Δ_{in}, while indices 6 through 10 correspond to the bit positions of the output difference Δ_{out}.
- **Polarity:** A positive integer i represents the literal x_i (the bit is 1), whereas a negative integer $-i$ represents the negated literal $\neg x_i$ (the bit is 0).
- **Logical Operation:** Each line is treated as a *disjunctive clause* (an OR relationship). The example clause above is mathematically expressed as:

$$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee \neg x_6 \vee x_7 \vee x_8 \vee x_9 \vee x_{10} \tag{B.3}$$

- **Termination:** The final 0 serves as a delimiter indicating the end of the clause.

Under this framework, the clause is violated (evaluated as False) if and only if all literals are false. In the provided example, the constraint effectively excludes the specific bit pattern where:

$$(x_1, x_2, x_3, x_4, x_5, x_7, x_8, x_9, x_{10}) = 0 \quad \text{and} \quad x_6 = 1 \tag{B.4}$$

By aggregating these constraints, the solver can prune the search space to include only valid differential transitions where the DDT entry is non-zero.

B.4 Solver Execution

The generated `attack.cnf` file can be solved using **CryptoMiniSat**, a high-performance SAT solver. The tool is open-source and can be downloaded from the official repository:

`https://github.com/msoos/cryptominisat/releases`

To find multiple valid output differentials for a given input, run the following command in the terminal:

```
cryptominisat5.exe --maxsol 20 attack.cnf
```

Appendix C

Target Difference Algorithm (TDA) Implementation

C.1 C++ Implementation of Matrix Inverse and State Generation

In addition to extracting the target state into the array U, the implementation now exports the fully reduced augmented matrix to an external file (`D.txt`). After performing Gaussian elimination over GF(2), the resulting 1600×1601 matrix (`D_out`) encapsulates the simplified system of linear equations.

Specifically, this matrix explicitly defines the linear constraints and algebraic relationships of the intermediate state y following the linear layers (θ, ρ, π) of the Keccak permutation. By exporting this relationship matrix, we can systematically trace the deterministic bit transitions and further integrate these linear constraints into our cryptanalysis models or SAT solvers.

Listing C.1: Construct U and Export Linear Relationships for y

```
1
2  #include <iostream>
3  #include <vector>
4  #include <cstdint>
5  #include <fstream>
6  #include <algorithm>
7
8  using namespace std;
9  using MatrixType2D = vector<vector<int>>;
10 using MatrixType3D = vector<vector<vector<int>>>;
```

```cpp

// === [Data Omitted] Predefined Cryptographic Constants ===
// Note: Full 1600-bit arrays and 32x32 lookup tables are abbreviated for brevity.

// Target output difference state (alpha_2)
const vector<int> target_alpha_2 = {
    0, 0, 0, 0, /* ... [Data omitted for brevity] ... */ 0, 0, 0, 0
};

// Difference Distribution Table (DDT) for Keccak's 5-bit Chi (S-box) function
MatrixType2D ddt_matrix = {
    {32, 0, 0, /* ... [Data omitted] ... */ 0, 0},
    // ... [Remaining rows omitted for brevity] ...
    {0, 2, 2, /* ... [Data omitted] ... */ 0, 2}
};

// Rho translation offsets
const int r_ro[5][5] = {
    { 0, 36,  3, 41, 18}, { 1, 44, 10, 45,  2}, {62,  6, 43, 15, 61},
    {28, 55, 25, 21, 56}, {27, 20, 39,  8, 14}
};

// Keccak Round Constants (RC) for Iota step
const uint64_t Keccak_RC[24] = {
    0x0000000000000001ULL, 0x0000000000008082ULL, 0x800000000000808AULL,
    /* ... [Intermediate constants omitted] ... */
    0x8000000080008008ULL
};

// === [Helper Functions] S-box Analysis ===

// Looks up the optimal input difference (beta_1) for a given output difference (alpha_2)
uint32_t findBestInputDifference(uint32_t out_diff) {
    int max_prob = 0;
    uint32_t best_in_diff = 0;
    for(int i = 0; i < 32; i++) {
        if(ddt_matrix[i][out_diff] > max_prob) {
            max_prob = ddt_matrix[i][out_diff];
            best_in_diff = i;
        }
    }
    return best_in_diff;
}

// Reshapes a 1600-bit 1D state into a 5x5x64 3D matrix
MatrixType3D buildStateMatrix(const vector<int>& m) {
    MatrixType3D B(5, vector<vector<int>>(5, vector<int>(64, 0)));
    for(int i = 0; i < 1600; i++) {
        B[(i / 64) % 5][i / 320][i % 64] = m[i];
    }
    return B;
}

// Derives the linear coefficient matrix (N) from the S-box conditions
MatrixType2D computeSboxLinearCoeffs(MatrixType3D B) {
    vector<vector<vector<vector<int>>>> C(5, vector<vector<vector<int>>>(5, vector<vector<int>>(5, vector<int>(64, 0))));
    MatrixType2D C_1(1600, vector<int>(1600, 0));
    for(int i = 0; i < 5; i++){
        for(int j = 0; j < 64; j++){
            C[0][1][i][j] = B[2][i][j]; C[0][2][i][j] = B[1][i][j];
            C[1][2][i][j] = B[3][i][j]; C[1][3][i][j] = B[2][i][j];
            C[2][3][i][j] = B[4][i][j]; C[2][4][i][j] = B[3][i][j];
            C[3][0][i][j] = B[4][i][j]; C[3][4][i][j] = B[0][i][j];
            C[4][0][i][j] = B[1][i][j]; C[4][1][i][j] = B[0][i][j];
```

```
        }
    }
    for(int i = 0; i < 5; i++)
        for(int j = 0; j < 5; j++)
            for(int k = 0; k < 5; k++)
                for(int l = 0; l < 64; l++)
                    C_1[320 * k + 64 * i + l][320 * k + 64 * j + l] = C[i][j][k][l];
    return C_1;
}

// Derives the constant terms of the S-box conditions
vector<int> computeSboxConstants(MatrixType3D B_1, MatrixType3D B_2) {
    MatrixType3D C_1(5, vector<vector<int>>(5, vector<int>(64, 0)));
    MatrixType3D C_2(5, vector<vector<int>>(5, vector<int>(64, 0)));
    vector<int> m(1600);

    for(int i = 0; i < 5; i++){
        for(int j = 0; j < 64; j++){
            C_2[0][i][j] = B_1[0][i][j] ^ (B_1[1][i][j] ^ 1) * B_1[2][i][j];
            C_2[1][i][j] = B_1[1][i][j] ^ (B_1[2][i][j] ^ 1) * B_1[3][i][j];
            C_2[2][i][j] = B_1[2][i][j] ^ (B_1[3][i][j] ^ 1) * B_1[4][i][j];
            C_2[3][i][j] = B_1[3][i][j] ^ (B_1[4][i][j] ^ 1) * B_1[0][i][j];
            C_2[4][i][j] = B_1[4][i][j] ^ (B_1[0][i][j] ^ 1) * B_1[1][i][j];
        }
    }
    for(int j = 0; j < 5; j++){
        for(int k = 0; k < 64; k++){
            C_1[0][j][k] = B_2[0][j][k] ^ C_2[0][j][k];
            C_1[1][j][k] = B_2[1][j][k] ^ C_2[1][j][k];
            C_1[2][j][k] = B_2[2][j][k] ^ C_2[2][j][k];
            C_1[3][j][k] = B_2[3][j][k] ^ C_2[3][j][k];
            C_1[4][j][k] = B_2[4][j][k] ^ C_2[4][j][k];
        }
    }
    for(int i = 0; i < 1600; i++) m[i] = C_1[(i / 64) % 5][i / 320][i % 64];
    return m;
}

// === [Helper Functions] Linear Layer Mapping ===

void theta(int in[5][5][64], int out[5][5][64]) {
    int C[5][64] = {0}, D[5][64] = {0};
    for(int x=0; x<5; x++)
        for(int z=0; z<64; z++)
            C[x][z] = in[x][0][z] ^ in[x][1][z] ^ in[x][2][z] ^ in[x][3][z] ^ in[x][4][z];

    for(int x=0; x<5; x++)
        for(int z=0; z<64; z++)
            D[x][z] = C[(x+4)%5][z] ^ C[(x+1)%5][(z+63)%64];

    for(int x=0; x<5; x++)
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++)
                out[x][y][z] = in[x][y][z] ^ D[x][z];
}

void rho(int in[5][5][64], int out[5][5][64]) {
    for(int x=0; x<5; x++)
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++) out[x][y][z] = in[x][y][(z - r_ro[x][y] + 64) % 64];
}

void pi(int in[5][5][64], int out[5][5][64]) {
    for(int x=0; x<5; x++)
```

```
        for(int y=0; y<5; y++)
            for(int z=0; z<64; z++) out[y][(2*x + 3*y)%5][z] = in[x][y][z];
}

// Generates the full 1600x1600 inverse mapping matrix for the linear layer
MatrixType2D getFullLInverseMatrix() {
    MatrixType2D L_matrix(1600, vector<int>(1600, 0));
    for(int i = 0; i < 1600; i++) {
        int S0[5][5][64] = {0}, S1[5][5][64] = {0}, S2[5][5][64] = {0}, S3[5][5][64] = {0};

        int y_in = i / 320, x_in = (i / 64) % 5, z_in = i % 64;
        S0[x_in][y_in][z_in] = 1;

        theta(S0, S1); rho(S1, S2); pi(S2, S3);

        for(int y = 0; y < 5; y++) {
            for(int x = 0; x < 5; x++) {
                for(int z = 0; z < 64; z++) {
                    L_matrix[320 * y + 64 * x + z][i] = S3[x][y][z];
                }
            }
        }
    }
    return L_matrix;
}

// Gaussian elimination over GF(2)
MatrixType2D Gaussian(MatrixType2D D) {
    int rows = D.size(), cols = D[0].size(), pivot_row = 0;
    for (int c = 0; c < cols - 1 && pivot_row < rows; c++) {
        int sel = pivot_row;
        while (sel < rows && D[sel][c] == 0) sel++;
        if (sel == rows) continue;
        if (sel != pivot_row) swap(D[pivot_row], D[sel]);

        for (int i = 0; i < rows; i++) {
            if (i != pivot_row && D[i][c] == 1) {
                for (int k = c; k < cols; k++) D[i][k] ^= D[pivot_row][k];
            }
        }
        pivot_row++;
    }
    return D;
}

// === [Main Routine] Merged Pipeline ===
int main() {
    // 1. Convert 1D alpha_2 array into 320 groups of 5 bits
    vector<int> alpha_2(320, 0), beta_1(320, 0);
    for(int i = 0; i < 320; i++){
        alpha_2[i] = target_alpha_2[i*5 + 0] + target_alpha_2[i*5 + 1]*2 +
                     target_alpha_2[i*5 + 2]*4 + target_alpha_2[i*5 + 3]*8 + target_alpha_2[i*5 + 4]*16;
    }

    // 2. Find Beta_1 backwards from Alpha_2 using DDT
    vector<int> Beta_1_1D(1600, 0);
    for(int i = 0; i < 320; i++){
        beta_1[i] = findBestInputDifference(alpha_2[i]);
        for(int j = 0; j < 5; j++) {
            Beta_1_1D[i*5 + j] = (beta_1[i] >> j) & 1;
        }
    }

    // 3. Build 3D states and compute constraints on z (Matrix N)
```

```
    MatrixType3D Alpha_2_3D = buildStateMatrix(target_alpha_2);
    MatrixType3D Beta_1_3D = buildStateMatrix(Beta_1_1D);

    MatrixType2D N_coeffs = computeSboxLinearCoeffs(Beta_1_3D);
    vector<int> N_consts = computeSboxConstants(Beta_1_3D, Alpha_2_3D);

    MatrixType2D N(1600, vector<int>(1601, 0));
    for(int i = 0; i < 1600; i++) {
        for(int j = 0; j < 1600; j++) N[i][j] = N_coeffs[i][j];
        N[i][1600] = N_consts[i];
    }

    // 4. Map z constraints back to y constraints using L_inv (Matrix D)
    MatrixType2D L_inv = getFullLInverseMatrix();
    MatrixType2D D(1600, vector<int>(1601, 0));
    for(int i = 0; i < 1600; i++) D[i][1600] = N[i][1600]; // Copy constants

    for(int i = 0; i < 1600; i++){
        for(int j = 0; j < 1600; j++){
            if(N[i][j] == 1){
                for(int k = 0; k < 1600; k++) D[i][k] ^= L_inv[j][k];
            }
        }
    }

    // 5. Apply Round Constant (Iota) Correction to the constants
    int target_round_idx = 0; // Configured for Round 0 RC
    uint64_t current_rc = Keccak_RC[target_round_idx];

    for(int i = 0; i < 1600; i++){
        int rc_dot_product = 0;
        for(int z = 0; z < 64; z++){
            if(D[i][z] == 1 && ((current_rc >> z) & 1)) rc_dot_product ^= 1;
        }
        D[i][1600] ^= rc_dot_product;
    }

    // 6. Simplify the constraints via Gaussian Elimination
    D = Gaussian(D);

    // 7. Compute array U: U[i]=0 if the entire row of variables is 0, else U[i]=1
    vector<int> U(1600, 0);
    for(int i = 0; i < 1600; i++){
        for(int j = 0; j < 1600; j++){
            if(D[i][j] == 1){
                U[i] = 1;
                break;
            }
        }
    }

    // 8. Output array U
    cout << "Array U:\n{";
    for(int i = 0; i < 1600; i++){
        cout << U[i] << (i == 1599 ? "" : ",");
    }
    cout << "}\n";

    // Write final D matrix to file for further processing
    ofstream outFile("D.txt");
    for(int i = 0; i < 1600; i++){
        for(int j = 0; j < 1601; j++) outFile << D[i][j] << " ";
        outFile << "\n";
    }
```

```
    outFile.close();

    return 0;
}
}
```

C.2 Preparation for Finding the 2-Round Connector

Before constructing the 2-round connector, it is essential to ensure that the differential trail is strictly consistent with the initial padding and capacity constraints. Crucially, the intermediate differential states, specifically β_0 and β_1, must be completely free of algebraic contradictions. To achieve this, Section C.2 details the procedure for deriving a valid β_1 from α_2, while Section C.3 presents the methodology for computing the corresponding β_0.

Listing C.2: Finding β_1

```
#include <iostream>
#include <vector>
#include <cstdint>

using namespace std;
using MatrixType2D = vector<vector<int>>;

// === [Data Omitted] Predefined Cryptographic Constants ===
// Target output difference state (alpha_2)
// Note: Full 1600-bit array is abbreviated for brevity.
const vector<int> target_alpha_2 = {
    0, 0, 0, 0, /* ... [Data omitted for brevity] ... */ 0, 0, 0, 0
};

// Difference Distribution Table (DDT) for Keccak's 5-bit Chi (S-box) function
MatrixType2D matrix = {
    {32, 0, 0, /* ... [Data omitted] ... */ 0, 0},
    // ... [Remaining 31 rows omitted for brevity] ...
    {0, 2, 2, /* ... [Data omitted] ... */ 0, 2}
};

// Reshapes a 1600-bit 1D state into a 320x5 2D matrix (grouped by 5-bit S-boxes)
MatrixType2D reshapeState(vector<int> m) {
    MatrixType2D n(m.size() / 5, vector<int>(5, 0));
    for(int i = 0; i < m.size(); i++)
        n[64 * (i / 320) + (i % 64)][(i / 64) % 5] = m[i];
    return n;
}

// Looks up the optimal input difference (beta_1) for a given output difference (alpha_2)
// Uses a greedy approach: selects the first input difference with the maximum probability.
uint32_t findBestInputDifference(uint32_t out_diff) {
    int max_prob = 0;
    uint32_t best_in_diff = 0;
    for(int i = 0; i < 32; i++) {
        if(matrix[i][out_diff] > max_prob) {
            max_prob = matrix[i][out_diff];
            best_in_diff = i;
        }
```

```
    }
    return best_in_diff;
}

int main() {
    // 1. Group the 1600-bit alpha_2 state into 320 5-bit lanes
    MatrixType2D Alpha_2 = reshapeState(target_alpha_2);
    vector<int> alpha_2(320, 0);
    vector<int> beta_1(320, 0);
    MatrixType2D Beta_1(320, vector<int>(5, 0));

    // 2. Convert each 5-bit binary array into a 32-bit integer for DDT lookup
    for(int i = 0; i < 320; i++){
        alpha_2[i] = Alpha_2[i][0] + Alpha_2[i][1]*2 + Alpha_2[i][2]*4 +
                     Alpha_2[i][3]*8 + Alpha_2[i][4]*16;
    }

    // 3. Backward extension: Find beta_1 from alpha_2 via DDT lookup
    for(int i = 0; i < 320; i++){
        beta_1[i] = findBestInputDifference(alpha_2[i]);
        // Convert the integer beta_1 back to a 5-bit binary representation
        for(int j = 0; j < 5; j++) {
            Beta_1[i][j] = (beta_1[i] >> j) & 1;
        }
    }

    // 4. Flatten the 320x5 beta_1 state back into a 1600-bit 1D array
    vector<int> Output(1600);
    for(int i = 0; i < 1600; i++) {
        Output[i] = Beta_1[64 * (i / 320) + (i % 64)][(i / 64) % 5];
    }

    // 5. Output the resulting Beta_1 state
    for(int i = 0; i < 1600; i++) {
        cout << Output[i] << (i == 1599 ? "" : ",");
    }

    return 0;
}
```

Listing C.3: Finding β_0

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <cmath>
#include <cstdint>
#include <algorithm>
#include <cstdlib>
#include <thread>
#include <chrono>

using namespace std;
using MatrixType2D = vector<vector<int>>;

// === [Definitions] Global Variables & CNF Clauses ===
int var_counter = 0;
vector<vector<int>> base_sbox_clauses; // Clauses for individual S-boxes
vector<vector<int>> linear_clauses;    // Clauses for the linear layer (Theta, Pi, Rho)
vector<vector<int>> blocking_clauses;  // Stores blocked beta_0 solutions to prevent re-evaluation

```

```cpp
// === [Data Omitted] Predefined Cryptographic Constants ===
// Note: Full 1600-bit arrays and lookup tables are abbreviated for brevity.
const vector<int> alpha_1 = {0, 0, 0, 0, /* ... [Data omitted for brevity] ... */ 1, 1, 1, 1};
const vector<int> beta_0 = {0, 0, 0, 0, /* ... [Data omitted for brevity] ... */ 1, 1, 0, 1};
const int alpha_2[1600] = {0, 1, 1, 1, /* ... [Data omitted for brevity] ... */ 1, 1, 1, 0};

MatrixType2D Alpha_1(320, vector<int>(5, 0));
MatrixType2D Beta_0(320, vector<int>(5, 0));

// DDT property lookup table for 5-bit S-box
MatrixType2D matrix_lookup = {
    {32, 0, 0, /* ... [Data omitted] ... */ 0, 0},
    // ... [Remaining rows omitted for brevity] ...
    {0, 2, 2, /* ... [Data omitted] ... */ 0, 2}
};

// Translation offsets for Rho and Pi steps
const int T[5][5] = {
    {0, 36, 3, 41, 18}, {1, 44, 10, 45, 2}, {62, 6, 43, 15, 61},
    {28, 55, 25, 21, 56}, {27, 20, 39, 8, 14}
};

// === [Helper Functions] Utility & Matrix Operations ===
inline int mod(int a, int b) { return (a % b + b) % b; }

int popcount(uint32_t n) {
    int count = 0;
    while (n) { count += n & 1; n >>= 1; }
    return count;
}

// Converts a boolean equation into CNF XOR clauses
void addXorAsCNF(vector<vector<int>>& clauses_list, const vector<uint32_t>& vars) {
    int n = vars.size();
    int num_combinations = 1 << n;
    for (int mask = 0; mask < num_combinations; ++mask) {
        if (popcount(mask) % 2 != 0) {
            vector<int> clause;
            for (int i = 0; i < n; ++i) {
                if ((mask >> i) & 1) clause.push_back(-vars[i]);
                else clause.push_back(vars[i]);
            }
            clauses_list.push_back(clause);
        }
    }
}

// Variable ID mappings for the SAT solver
// Alpha (Input): 1~1600, ColSum (Aux): 1601~1920, Beta (Output): 1921~3520
uint32_t getAlphaVar(int i, int j, int k) { return 1 + (320 * j + 64 * i + k); }
uint32_t getColSumVar(int i, int k)       { return 1601 + (64 * i + k); }
uint32_t getBetaVar(int i, int j, int k)  { return 1921 + (320 * j + 64 * i + k); }

// Generates CNF clauses for the Keccak Linear Layer
void precompute_linear_layer_clauses() {
    // 1. ColumnSum (Theta step part 1)
    for (int i = 0; i < 5; ++i) {
        for (int k = 0; k < 64; ++k) {
            vector<uint32_t> xor_vars = {getColSumVar(i, k)};
            for (int j = 0; j < 5; ++j) xor_vars.push_back(getAlphaVar(i, j, k));
            addXorAsCNF(linear_clauses, xor_vars);
        }
    }
    // 2. Beta mapping (Theta part 2, Pi, Rho steps)
```

```
    for (int i = 0; i < 5; ++i) {
        for (int j = 0; j < 5; ++j) {
            for (int k = 0; k < 64; ++k) {
                int mapped_i = j;
                int mapped_j = mod(2 * i + 3 * j, 5);
                int mapped_k = mod(k + T[i][j], 64);
                vector<uint32_t> xor_vars = {
                    getBetaVar(mapped_i, mapped_j, mapped_k),
                    getAlphaVar(i, j, k),
                    getColSumVar(mod(i - 1, 5), k),
                    getColSumVar(mod(i + 1, 5), mod(k - 1, 64))
                };
                addXorAsCNF(linear_clauses, xor_vars);
            }
        }
    }
}

// === [SAT Logic] S-box Constraints Generation ===
int newVar() { return ++var_counter; }
void add_clause(const vector<int>& lits) { base_sbox_clauses.push_back(lits); }

// Keccak Chi (Non-linear) function
uint32_t chi(uint32_t in) {
    uint32_t out = 0, x[5];
    for(int i=0; i<5; i++) x[i] = (in >> i) & 1;
    for(int i=0; i<5; i++) {
        uint32_t bit = x[i] ^ ((x[(i+1)%5] ^ 1) & x[(i+2)%5]);
        out |= (bit << i);
    }
    return out;
}

// Verifies if a differential transition through Chi is possible
bool is_valid_differential(uint32_t din, uint32_t dout) {
    for(uint32_t x = 0; x < 32; x++) {
        if ((chi(x) ^ chi(x ^ din)) == dout) return true;
    }
    return false;
}

// Precomputes all invalid transitions for a 5-bit S-box
void precompute_base_sbox_constraints() {
    vector<int> din_vars, dout_vars;
    for(int i=0; i<5; i++) din_vars.push_back(newVar());
    for(int i=0; i<5; i++) dout_vars.push_back(newVar());

    for(uint32_t d_in = 0; d_in < 32; d_in++) {
        for(uint32_t d_out = 0; d_out < 32; d_out++) {
            if (!is_valid_differential(d_in, d_out)) {
                vector<int> clause;
                for(int i=0; i<5; i++) clause.push_back(((d_in >> i) & 1) ? -din_vars[i] : din_vars[i]);
                for(int i=0; i<5; i++) clause.push_back(((d_out >> i) & 1) ? -dout_vars[i] : dout_vars[i]);
                add_clause(clause);
            }
        }
    }
}

// Maps 1D index to SAT variable space (Beta output range)
int f_idx(int x){ return (x <= 5) ? (1920 + x*64 - 63) : (3520 + (x - 5)*64 - 63); }

// Constructs the final CNF file for CryptoMiniSat
void generate_attack1600_cnf(const vector<vector<int>>& blocks) {
```

```
    int total_vars = 5120;
    int total_clauses = (base_sbox_clauses.size() * 320) + 1600 + 262 + linear_clauses.size() + blocks.size();

    ofstream outfile("attack1600.cnf");
    outfile << "p␣cnf␣" << total_vars << "␣" << total_clauses << "\n";

    for(const auto& cls : linear_clauses) {
        for(int lit : cls) outfile << lit << "␣";
        outfile << "0\n";
    }

    for(int y=0; y<5; y++){
        for(int z=0; z<64; z++){
            for(const auto& cls : base_sbox_clauses) {
                for(int lit : cls) outfile << ((lit < 0) ? -1 : 1) * (f_idx(abs(lit)) + 320*y + z) << "␣";
                outfile << "0\n";
            }
        }
    }

    // Output alpha_2 targets
    for(int i = 0; i < 1600; i++) {
        outfile << ((alpha_2[i] == 1) ? (3521 + i) : -(3521 + i)) << "␣0\n";
    }
    // Padding constraints (zeroes for variables 1339~1600)
    for(int i = 1338; i < 1600; i++) outfile << -(1 + i) << "␣0\n";

    // Add blocking clauses to exclude previously found invalid beta_0
    for(const auto& b_cls : blocks) {
        for(int lit : b_cls) outfile << lit << "␣";
        outfile << "0\n";
    }
    outfile.close();
}

// Executes SAT solver and parses the output for Beta_0
bool run_sat_solver_and_get_beta0(vector<int>& current_beta_0) {
    bool is_sat = false;
    for (int attempt = 1; attempt <= 5; ++attempt) {
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        system("cryptominisat5␣attack1600.cnf␣>␣sat_result.txt");

        ifstream infile("sat_result.txt");
        if (!infile.is_open()) {
            std::this_thread::sleep_for(std::chrono::milliseconds(500));
            continue;
        }

        string line;
        bool valid_output = false;
        current_beta_0.assign(1600, 0);

        while (getline(infile, line)) {
            if (line.rfind("s␣", 0) == 0) {
                valid_output = true;
                if (line.find("SATISFIABLE") != string::npos && line.find("UNSATISFIABLE") == string::npos) is_sat = true;
            } else if (line.rfind("v␣", 0) == 0) {
                stringstream ss(line.substr(2));
                int var;
                while (ss >> var) {
                    if (var == 0) continue;
                    if (abs(var) >= 1921 && abs(var) <= 3520) current_beta_0[abs(var) - 1921] = (var > 0) ? 1 : 0;
                }
            }
```

```
        }
        infile.close();
        if (valid_output) return is_sat;
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }
    exit(1); // Abort after 5 failures
}

// === [Linear Algebra] Matrix Deductions ===
void VtoM(const vector<int>& v, MatrixType2D& m){
    for(int i = 0; i < 1600; i++) m[64*(i/320)+(i%64)][(i/64)%5] = v[i];
}

vector<uint32_t> UINT32(int i){
     uint32_t AL = Alpha_1[i][0] + Alpha_1[i][1]*2 + Alpha_1[i][2]*4 + Alpha_1[i][3]*8 + Alpha_1[i][4]*16;
     uint32_t BE = Beta_0[i][0] + Beta_0[i][1]*2 + Beta_0[i][2]*4 + Beta_0[i][3]*8 + Beta_0[i][4]*16;
     return {AL, BE};
}

// Derives linear conditions for a given S-box
MatrixType2D CC(uint32_t m, uint32_t n) {
    MatrixType2D c(5, vector<int>(6, 0));
    int x[5], y[5];
    for(int i = 0; i < 5; i++) { x[i] = (m >> i) & 1; y[i] = (n >> i) & 1; }
    for(int i = 0; i < 5; i++) c[i][5] = y[i] ^ (x[i] ^ ((x[(i + 1) % 5] ^ 1) * x[(i + 2) % 5]));
    c[0][1] = x[2]; c[0][2] = x[1]; c[1][2] = x[3]; c[1][3] = x[2];
    c[2][3] = x[4]; c[2][4] = x[3]; c[3][0] = x[4]; c[3][4] = x[0];
    c[4][0] = x[1]; c[4][1] = x[0];
    return c;
}

// Simplifies differential conditions to linear constraints (DDT=8)
MatrixType2D DDT8(uint32_t m, uint32_t n, vector<bool>& is_nonlinear, int sbox_index) {
    MatrixType2D c = CC(m, n);
    MatrixType2D L(5, vector<int>(6, 0));
    int val[5] = {-1, -1, -1, -1, -1};

    for(int i = 0; i < 5; i++) {
        if (c[i][(i+1)%5] + c[i][(i+2)%5] == 1) {
            val[(c[i][(i+1)%5] == 1) ? (i+1)%5 : (i+2)%5] = c[i][5];
        }
    }

    for(int k = 0; k < 5; k++) {
        L[k][k] = 1; // Preserve base variable Y_k = X_k
        int v1 = val[(k+1)%5], v2 = val[(k+2)%5];

        if (v1 != -1 && v2 != -1) L[k][5] ^= ((v1 ^ 1) & v2);
        else if (v1 == 0) L[k][(k+2)%5] ^= 1;
        else if (v2 == 1) { L[k][(k+1)%5] ^= 1; L[k][5] ^= 1; }
        else if (c[k][(k+1)%5] == 1 && c[k][(k+2)%5] == 1) {
            if (c[k][5] == 1) L[k][(k+2)%5] ^= 1;
        } else {
            // Irreducible non-linear term found, poison this S-box
            is_nonlinear[320*(sbox_index/64) + (sbox_index % 64) + 64*k] = true;
            for(int j = 0; j < 6; j++) L[k][j] = 0;
        }
    }
    return L;
}

// Gaussian elimination over GF(2)
bool Gaussian(MatrixType2D D){
    int rows = D.size(), cols = D[0].size(), pivot_row = 0;
```

```
    for (int c = 0; c < cols - 1 && pivot_row < rows; c++) {
        int sel = pivot_row;
        while (sel < rows && D[sel][c] == 0) sel++;
        if (sel == rows) continue;
        if (sel != pivot_row) swap(D[pivot_row], D[sel]);

        for (int i = 0; i < rows; i++) {
            if (i != pivot_row && D[i][c] == 1) {
                for (int k = c; k < cols; k++) D[i][k] ^= D[pivot_row][k];
            }
        }
        pivot_row++;
    }
    // Check for contradiction (0 = 1)
    for (int i = pivot_row; i < rows; i++) if (D[i][cols - 1] == 1) return false;
    return true;
}

// Substitutes non-linear elements and updates the global constraint matrix
MatrixType2D Substitution(const MatrixType2D& M, const MatrixType2D& N, const vector<bool>& is_nonlinear) {
    MatrixType2D m(M.size(), vector<int>(M[0].size(), 0));
    for (size_t i = 0; i < M.size(); i++) {
        bool equation_is_poisoned = false;
        for (int j = 0; j < 1600; j++) {
            if (M[i][j] == 1 && is_nonlinear[j]) { equation_is_poisoned = true; break; }
        }
        if (equation_is_poisoned) continue;
        for (int j = 0; j < 1600; j++) {
            if (M[i][j] == 1) for (int k = 0; k < 1601; k++) m[i][k] ^= N[j][k];
        }
        m[i][1600] ^= M[i][1600];
    }
    return m;
}

MatrixType2D Merge(const MatrixType2D& M, const MatrixType2D& N){
    MatrixType2D m(M.size() + N.size(), vector<int>(M[0].size()));
    for(size_t i = 0; i < M.size(); i++) m[i] = M[i];
    for(size_t i = M.size(); i < M.size() + N.size(); i++) m[i] = N[i - M.size()];
    return m;
}

// === [Main Routine] Iterative SAT & Gaussian Verification ===
int main() {
    precompute_base_sbox_constraints();
    precompute_linear_layer_clauses();

    // 1. Load absolute physical constraints (E_M and D matrices)
    int rows = 262, cols = 1601;
    MatrixType2D E_M(rows, vector<int>(cols));
    ifstream inFile("E_M.txt");
    if (inFile) { for (int i = 0; i < rows; ++i) for (int j = 0; j < cols; ++j) inFile >> E_M[i][j]; inFile.close(); }

    int rows_1 = 1600, cols_1 = 1601;
    MatrixType2D D(rows_1, vector<int>(cols_1));
    ifstream inFileD("D.txt");
    if (inFileD) { for (int i = 0; i < rows_1; ++i) for (int j = 0; j < cols_1; ++j) inFileD >> D[i][j]; inFileD.close();
         }

    VtoM(alpha_1, Alpha_1);

    bool success = false;
    for (int attempt = 1; attempt <= 100; ++attempt) {
        // 2. Generate CNF & Execute SAT Solver
```

```
        generate_attack1600_cnf(blocking_clauses);
        vector<int> current_beta_0;
        if (!run_sat_solver_and_get_beta0(current_beta_0)) break; // UNSAT

        // 3. Matrix preparation for Gaussian Check
        vector<bool> is_nonlinear(1600, false);
        VtoM(current_beta_0, Beta_0);
        MatrixType2D L1600(1600, vector<int>(1601, 0)), C(1600, vector<int>(1601, 0));

        for(int i = 0; i < 320; i++){
            vector<uint32_t> UU = UINT32(i);
            MatrixType2D C_1 = CC(UU[1], UU[0]);
            for(int j = 0; j < 5; j++) for(int k = 0; k < 5; k++) C[320*(i/64)+(i%64)+64*j][320*(i/64)+(i%64)+64*k] = C_1[
                j][k];
            for(int j = 0; j < 5; j++) C[320*(i/64)+(i%64)+64*j][1600] = C_1[j][5];
        }

        // 4. DDT Analysis and Nonlinearity Poisoning
        bool invalid_ddt = false;
        for(int i = 0; i < 320; i++){
            vector<uint32_t> UU = UINT32(i);
            int DDT = matrix_lookup[UU[1]][UU[0]];
            MatrixType2D L(5, vector<int>(6, 0));

            if(DDT == 2 || DDT == 4 || DDT == 8) {
                L = DDT8(UU[1], UU[0], is_nonlinear, i);
            } else if(DDT == 32) {
                for(int j = 0; j < 5; j++) is_nonlinear[320*(i/64) + (i% 64) + 64*j] = true;
            } else {
                invalid_ddt = true; break;
            }

            for(int j = 0; j < 5; j++) for(int k = 0; k < 5; k++) L1600[320*(i/64)+(i%64)+64*j][320*(i/64)+(i%64)+64*k] =
                L[j][k];
            for(int j = 0; j < 5; j++) L1600[320*(i/64)+(i%64)+64*j][1600] = L[j][5];
        }

        if (!invalid_ddt) {
            // 5. Gaussian Elimination Verification
            MatrixType2D E_M_1 = Substitution(D, L1600, is_nonlinear);
            MatrixType2D E_M_temp_1 = Merge(E_M_1, C);

            if(Gaussian(E_M_temp_1)) {
                E_M = E_M_temp_1;
                success = true;
                break; // A consistent Beta_0 is found!
            }
        }

        // 6. Conflict detected, add to blocking clauses (De Morgan's Law)
        vector<int> b_clause;
        for(int i = 0; i < 1600; i++) {
            int var_id = 1921 + i;
            b_clause.push_back(current_beta_0[i] == 1 ? -var_id : var_id);
        }
        blocking_clauses.push_back(b_clause);
    }

    if (success) {
        ofstream outEM("Updated_E_M.txt");
        for(size_t i = 0; i < E_M.size(); i++){
            for(size_t j = 0; j < E_M[i].size(); j++) outEM << E_M[i][j] << (j == E_M[i].size() - 1 ? "" : "␣");
            outEM << "\n";
        }
```

```
403             outEM.close();
404         }
405         return 0;
406     }
```

C.3 2-Round Connector

Detailed Explanation of Listing The provided program implements an automated, highly optimized algebraic linearization tool designed to analyze the non-linear χ transformation layer in round-reduced Keccak permutations. Its primary objective is to resolve the quadratic relations introduced by the 5-bit S-boxes over $\mathbb{F}_2$ and accumulate valid linear constraints into a global system matrix E_M.

To optimize computational efficiency, the algorithm deterministically evaluates the differential constraints applied to each local S-box. The linearization strategy depends strictly on its Difference Distribution Table (DDT) entry:

1. **Deterministic Linearization ($DDT \in \{2, 4, 8\}$):** Processed via `DDT8`, the algorithm systematically extracts all constant conditions derived from the S-box differences. By applying these definitive constant bounds, quadratic equations are safely reduced to linear equations without requiring external guessing. Irreducible non-linear terms are explicitly identified, isolated, and marked in an `is_nonlinear` tracking array to avoid poisoning the linear system.

2. **Purely Linear State ($DDT = 32$):** Occurring under zero-difference conditions, the algorithm randomly retains three independent base constraints to preserve system dimensionality and global connectivity while discarding the remaining relations to maintain sufficient degrees of freedom.

To construct a strictly valid global trail, the algorithm seamlessly transitions into a Monte Carlo simulation phase to resolve the isolated irreducible non-linear terms. Specifically, the algorithm randomly assigns valid physical assumptions (binary constraints) to the unresolved non-linear variables and strictly substitutes these conditions back into the dependency matrix. The fully constrained 1600-dimensional system is then aggressively re-evaluated. This stochastic guessing process is iterated until a perfect global match—entirely free of algebraic contradictions—is discovered.

Crucially, the integration of on-the-fly Gaussian elimination serves as an immediate consistency check. By evaluating the augmented column of the merged matrix, the algorithm instantly detects algebraic contradictions ($0 = 1$). This mechanism efficiently discards incompatible differential trails on the spot. Upon finding an algebraically consistent pathway, the updated matrix E_M is exported to `Perfect_E_M.txt` for subsequent cryptanalysis.

Listing C.4: Finding 2-round Connector via Monte Carlo Simulation

```
#include <iostream>
#include <vector>
#include <cstdint>
#include <fstream>
#include <algorithm>
#include <random>
#include <chrono>

using namespace std;
using MatrixType2D = vector<vector<int>>;

// ============================================================================
// === Constant Definitions (Omitted for thesis brevity)                    ===
// ============================================================================
vector<int> alpha_1 = { /* ... 1600 bits omitted for brevity ... */ };
vector<int> beta_0  = { /* ... 1600 bits omitted for brevity ... */ };
MatrixType2D matrix = { /* ... 32x32 DDT omitted for brevity ... */ };

MatrixType2D Alpha_1(320, vector<int>(5, 0));
MatrixType2D Beta_0(320, vector<int>(5, 0));

void VtoM(const vector<int>& v, MatrixType2D& m){
    for(int i = 0; i < 1600; i++) m[64*(i/320) + (i%64)][(i/64)%5] = v[i];
}

vector<uint32_t> UINT32(int i){
     uint32_t AL = Alpha_1[i][0] + Alpha_1[i][1]*2 + Alpha_1[i][2]*4 + Alpha_1[i][3]*8 + Alpha_1[i][4]*16;
     uint32_t BE = Beta_0[i][0] + Beta_0[i][1]*2 + Beta_0[i][2]*4 + Beta_0[i][3]*8 + Beta_0[i][4]*16;
     return {AL, BE};
}

MatrixType2D CC(uint32_t m, uint32_t n) {
    MatrixType2D c(5, vector<int>(6, 0));
    int x[5], y[5];
    for(int i = 0; i < 5; i++) { x[i] = (m >> i) & 1; y[i] = (n >> i) & 1; }
    for (int i = 0; i < 5; i++) c[i][5] = y[i] ^ (x[i] ^ ((x[(i + 1) % 5] ^ 1) * x[(i + 2) % 5]));
    c[0][1] = x[2]; c[0][2] = x[1]; c[1][2] = x[3]; c[1][3] = x[2];
    c[2][3] = x[4]; c[2][4] = x[3]; c[3][0] = x[4]; c[3][4] = x[0];
    c[4][0] = x[1]; c[4][1] = x[0];
    return c;
}

bool Gaussian(MatrixType2D D){
    int rows = D.size(), cols = D[0].size(), pivot_row = 0;
    for (int c = 0; c < cols - 1 && pivot_row < rows; c++) {
        int sel = pivot_row;
        while (sel < rows && D[sel][c] == 0) sel++;
        if (sel == rows) continue;
        if (sel != pivot_row) swap(D[pivot_row], D[sel]);
        for (int i = 0; i < rows; i++) {
            if (i != pivot_row && D[i][c] == 1) {
                for (int k = c; k < cols; k++) D[i][k] ^= D[pivot_row][k];
```

```
            }
        }
        pivot_row++;
    }
    for (int i = pivot_row; i < rows; i++) if (D[i][cols - 1] == 1) return false;
    return true;
}

// Deterministic linearization evaluating constant tracking
MatrixType2D DDT8(uint32_t m, uint32_t n, vector<bool>& is_nonlinear, int sbox_index) {
    MatrixType2D c = CC(m, n);
    MatrixType2D L(5, vector<int>(6, 0));
    int val[5] = {-1, -1, -1, -1, -1}; // Record deterministic constant values

    // Step 1: Collect all constant conditions for this S-box
    for(int i = 0; i < 5; i++) {
        int sum_c = c[i][(i+1)%5] + c[i][(i+2)%5];
        if (sum_c == 1) {
            if (c[i][(i+1)%5] == 1) val[(i+1)%5] = c[i][5];
            else val[(i+2)%5] = c[i][5];
        }
    }

    // Step 2: Preserve base variable and linearize the NL terms
    for(int k = 0; k < 5; k++) {
        L[k][k] = 1; // Always preserve base variable Y_k = X_k

        int v1 = val[(k+1)%5];
        int v2 = val[(k+2)%5];

        // Evaluate non-linear term NL_k = (X_{k+1} ^ 1) * X_{k+2}
        if (v1 != -1 && v2 != -1) {
            L[k][5] ^= ((v1 ^ 1) & v2); // Both are constants, NL becomes a pure constant
        } else if (v1 != -1) {
            if (v1 == 0) L[k][(k+2)%5] = 1; // Only v1 is constant
        } else if (v2 != -1) {
            if (v2 == 1) { // Only v2 is constant
                L[k][(k+1)%5] = 1;
                L[k][5] = 1;
            }
        } else if (c[k][(k+1)%5] == 1 && c[k][(k+2)%5] == 1) {
            int C = c[k][5]; // Both bound by current equation
            if (C == 1) L[k][(k+2)%5] = 1;
        } else {
            // Completely irreducible, mark as nonlinear and clear row
            is_nonlinear[320*(sbox_index/64) + (sbox_index % 64) + 64*k] = true;
            for(int j = 0; j < 6; j++) L[k][j] = 0;
        }
    }
    return L;
}

// Substitution ignoring poisoned (irreducible) non-linear terms for base evaluation
MatrixType2D Substitution_WithPoison(const MatrixType2D& M, const MatrixType2D& N, const vector<bool>& is_nonlinear) {
    MatrixType2D m;
    for (size_t i = 0; i < M.size(); i++) {
        bool equation_is_poisoned = false;
        for (int j = 0; j < 1600; j++) {
            if (M[i][j] == 1 && is_nonlinear[j]) { equation_is_poisoned = true; break; }
        }
        if (equation_is_poisoned) continue;
        vector<int> row(1601, 0);
        for (int j = 0; j < 1600; j++) {
            if (M[i][j] == 1) for (int k = 0; k < 1601; k++) row[k] ^= N[j][k];
```

```
        }
        row[1600] ^= M[i][1600];
        m.push_back(row);
    }
    return m;
}

// Strict substitution evaluating all constraints globally
MatrixType2D Substitution_NoDrop(const MatrixType2D& M, const MatrixType2D& N) {
    MatrixType2D m(M.size(), vector<int>(1601, 0));
    for (size_t i = 0; i < M.size(); i++) {
        for (int j = 0; j < 1600; j++) {
            if (M[i][j] == 1) for (int k = 0; k < 1601; k++) m[i][k] ^= N[j][k];
        }
        m[i][1600] ^= M[i][1600];
    }
    return m;
}

MatrixType2D Merge(MatrixType2D M, const MatrixType2D& N){
    M.insert(M.end(), N.begin(), N.end());
    return M;
}

int main(){
    int rows = 262, cols = 1601;
    MatrixType2D E_M_base(rows, vector<int>(cols));
    ifstream inFile("E_M.txt");
    if (inFile) { for (int i = 0; i < rows; ++i) for (int j = 0; j < cols; ++j) inFile >> E_M_base[i][j]; inFile.close();
         }

    int rows_1 = 1600, cols_1 = 1601;
    MatrixType2D D(rows_1, vector<int>(cols_1));
    ifstream inFileD("D.txt");
    if (inFileD) { for (int i = 0; i < rows_1; ++i) for (int j = 0; j < cols_1; ++j) inFileD >> D[i][j]; inFileD.close();
         }

    VtoM(alpha_1, Alpha_1); VtoM(beta_0, Beta_0);

    // Explicitly generate forward constraints and fuse them into E_M_base globally
    MatrixType2D C_matrix;
    for(int i = 0; i < 320; i++){
        vector<uint32_t> UU = UINT32(i);
        MatrixType2D C_1 = CC(UU[1], UU[0]);
        for(int j = 0; j < 5; j++) {
            vector<int> row(1601, 0);
            for(int k = 0; k < 5; k++) row[320*(i/64) + (i%64) + 64*k] = C_1[j][k];
            row[1600] = C_1[j][5];
            C_matrix.push_back(row);
        }
    }
    E_M_base = Merge(E_M_base, C_matrix);

    unsigned seed = std::chrono::system_clock::now().time_since_epoch().count();
    mt19937 g(seed);

    MatrixType2D best_E_M_temp, best_L1600;
    vector<bool> best_is_nonlinear;

    // Initialization: Establish the linear constraints and isolate irreducible terms
    MatrixType2D L1600(1600, vector<int>(1601, 0));
    vector<bool> is_nonlinear(1600, false);
    MatrixType2D current_E_M = E_M_base;

```

```
    for(int i = 0; i < 320; i++){
        vector<uint32_t> UU = UINT32(i);
        int DDT = matrix[UU[1]][UU[0]];
        MatrixType2D L(5, vector<int>(6, 0));

        if(DDT == 2 || DDT == 4 || DDT == 8) {
            L = DDT8(UU[1], UU[0], is_nonlinear, i);
        }
        else if(DDT == 32){
            vector<int> indices = {0, 1, 2, 3, 4};
            shuffle(indices.begin(), indices.end(), g);
            for(int d = 0; d < 3; d++) L[indices[d]][indices[d]] = 1;
            for(int d = 3; d < 5; d++) is_nonlinear[320*(i/64) + (i%64) + 64*indices[d]] = true;
        }

        for(int j = 0; j < 5; j++) {
            int row_idx = 320*(i/64) + (i%64) + 64*j;
            for(int k = 0; k < 5; k++) L1600[row_idx][320*(i/64) + (i%64) + 64*k] = L[j][k];
            L1600[row_idx][1600] = L[j][5];
        }
    }

    best_E_M_temp = current_E_M;
    best_L1600 = L1600;
    best_is_nonlinear = is_nonlinear;

    // Execution: Monte Carlo Simulation Guesses
    std::uniform_int_distribution<> distrib_option(0, 5);
    bool found_perfect_match = false;
    MatrixType2D perfect_E_M_temp;

    for (int guess = 1; guess <= 10000; guess++) {
        MatrixType2D L1600_guess = best_L1600;
        MatrixType2D E_M_guess = best_E_M_temp;

        // Substitute assumptions into E_M and apply resolved relations to Y
        for (int I = 0; I < 320; I++) {
            for (int b = 0; b < 5; b++) {
                int g_idx = 320*(I/64) + (I%64) + 64*b;

                if (best_is_nonlinear[g_idx]) {
                    int col1 = 320*(I/64) + (I%64) + 64*((b+1)%5);
                    int col2 = 320*(I/64) + (I%64) + 64*((b+2)%5);
                    int opt = distrib_option(g);

                    vector<int> eq(1601, 0);
                    L1600_guess[g_idx][g_idx] = 1; // Base relation y_b = x_b

                    if (opt == 0) {
                        eq[col1] = 1; eq[1600] = 1;
                    } else if (opt == 1) {
                        eq[col1] = 1; eq[1600] = 0;
                        L1600_guess[g_idx][col2] = 1;
                    } else if (opt == 2) {
                        eq[col2] = 1; eq[1600] = 1;
                        L1600_guess[g_idx][col1] = 1; L1600_guess[g_idx][1600] = 1;
                    } else if (opt == 3) {
                        eq[col2] = 1; eq[1600] = 0;
                    } else if (opt == 4) {
                        eq[col1] = 1; eq[col2] = 1; eq[1600] = 1;
                        L1600_guess[g_idx][col2] = 1;
                    } else if (opt == 5) {
                        eq[col1] = 1; eq[col2] = 1; eq[1600] = 0;
                    }
```

```
                    E_M_guess.push_back(eq); // Enforce assumption explicitly
                }
            }
        }

        // Substitute resolved Y equations into D globally
        MatrixType2D E_M_1_guess = Substitution_NoDrop(D, L1600_guess);
        MatrixType2D E_M_temp_guess = Merge(E_M_1_guess, E_M_guess);

        if (Gaussian(E_M_temp_guess)) {
            found_perfect_match = true;
            perfect_E_M_temp = E_M_temp_guess;
            break;
        }
    }

    if (found_perfect_match) {
        ofstream outEM("Perfect_E_M.txt");
        for(size_t i = 0; i < perfect_E_M_temp.size(); i++){
            for(size_t j = 0; j < perfect_E_M_temp[i].size(); j++){
                outEM << perfect_E_M_temp[i][j] << (j == perfect_E_M_temp[i].size() - 1 ? "" : "␣");
            }
            outEM << "\n";
        }
        outEM.close();
    }
    return 0;
}
```

Appendix D

Target Internal Difference Algorithm (TIDA)

D.1 Finding the Internal Differential Trail via SAT

The provided modeling program demonstrates the construction of the Boolean satisfiability (SAT) constraints utilized to search for the optimal internal differential trail. To minimize the probability of algebraic contradictions during the subsequent message selection and trail connection phases, we impose strict structural and weight bounds on the differential propagation. Specifically, we enforce that the initial state difference α_1 strictly resides within the Column-Parity (CP) kernel to prevent early diffusion. Furthermore, we restrict the total number of active S-boxes across the entire trail to a maximum of 20, and strictly limit the propagation weight of the intermediate state β_1 to be no greater than 7. These stringent conditions ensure the discovery of an internal differential characteristic with a sufficiently high transition probability.

Once the Conjunctive Normal Form (CNF) formula is fully generated by our modeling program, we employ the state-of-the-art SAT solver CryptoMiniSat5 to find the satisfying assignments and recover the target internal difference. The optimal internal differential trail recovered through this automated search methodology exactly matches the high-probability characteristic previously presented in [6].

Listing D.1: SAT-based Cryptanalysis Modeling for Keccak Target Internal Difference Algo-

rithm (TIDA)

```cpp
#include <iostream>
#include <vector>
#include <cstdint>
#include <fstream>
#include <cmath>

using namespace std;

class CNFWriter {
public:
    vector<vector<int>> clauses;
    int max_var = 0;
    void add_clause(const vector<int>& c) {
        clauses.push_back(c);
        for (int lit : c) max_var = max(max_var, abs(lit));
    }
    void write(const string& fn) {
        ofstream out(fn);
        out << "p␣cnf␣" << max_var << "␣" << clauses.size() << "\n";
        for (const auto& c : clauses) {
            for (int lit : c) out << lit << "␣";
            out << "0\n";
        }
    }
};

inline int mod(int a, int b) { return (a % b + b) % b; }
int popcnt(uint32_t n) { int c = 0; while (n) { c += n & 1; n >>= 1; } return c; }

const uint64_t RC[24] = { 0x0000000000000001ULL, 0x0000000000008082ULL, 0x800000000000808aULL };
const int T[5][5] = { {0, 36, 3, 41, 18}, {1, 44, 10, 45, 2}, {62, 6, 43, 15, 61},
                      {28, 55, 25, 21, 56}, {27, 20, 39, 8, 14} };

// ========================================
// 1D Flat Index Mapping (Lane-major: FIPS 202)
// ========================================
inline int flat_id(int i, int j, int k) { return (j * 5 + i) * 64 + k; }
// r = round index, mapping variables consistently across rounds
int v_a(int r, int i, int j, int k) { return 1 + r*4800 + flat_id(i,j,k); }      // alpha
int v_b(int r, int i, int j, int k) { return 1601 + r*4800 + flat_id(i,j,k); }   // beta
int v_g(int r, int i, int j, int k) { return 3201 + r*4800 + flat_id(i,j,k); }   // gamma
int v_cs(int r, int i, int k)       { return 12801 + r*320 + i*64 + k; }         // colsum
int v_act(int r, int j, int k)      { return 13761 + r*320 + j*64 + k; }         // active sbox

int aux_var = 14500;

// Sequential Counter Encoding for limiting total Hamming weight
void add_weight_limit(CNFWriter& w, const vector<int>& vars, int K) {
    int n = vars.size();
    if (n <= K) return;
    auto s = [&](int i, int j) { return aux_var + i * K + j - 1; };
    aux_var += n * K;
    w.add_clause({-vars[0], s(0, 1)});
    for (int i = 1; i < n; ++i) {
        w.add_clause({-vars[i], s(i, 1)});
        w.add_clause({-s(i-1, 1), s(i, 1)});
        for (int j = 2; j <= K; ++j) {
            w.add_clause({-s(i-1, j), s(i, j)});
            w.add_clause({-vars[i], -s(i-1, j-1), s(i, j)});
        }
        w.add_clause({-vars[i], -s(i-1, K)});
    }
}
```

```cpp

// Convert XOR constraints to CNF clauses
void add_xor(CNFWriter& w, const vector<int>& vars) {
    int n = vars.size(), combos = 1 << n;
    for (int mask = 0; mask < combos; ++mask) {
        if (popcnt(mask) % 2 != 0) {
            vector<int> c;
            for (int i = 0; i < n; ++i) c.push_back(((mask >> i) & 1) ? -vars[i] : vars[i]);
            w.add_clause(c);
        }
    }
}

// ==========================================
// Keccak Component Encodings
// ==========================================
void build_linear_layer(CNFWriter& w, int r) {
    for (int i = 0; i < 5; ++i) for (int k = 0; k < 64; ++k) {
        vector<int> xor_c = {v_cs(r, i, k)};
        for (int j = 0; j < 5; ++j) xor_c.push_back(v_a(r, i, j, k));
        add_xor(w, xor_c); // Compute Column Parity
    }
    for (int i = 0; i < 5; ++i) for (int j = 0; j < 5; ++j) for (int k = 0; k < 64; ++k) {
        vector<int> xor_v = {
            v_b(r, j, mod(2*i+3*j, 5), mod(k+T[i][j], 64)),
            v_a(r, i, j, k), v_cs(r, mod(i-1, 5), k), v_cs(r, mod(i+1, 5), mod(k-1, 64))
        };
        add_xor(w, xor_v); // Theta + Rho + Pi mapping
    }
}

void build_sbox_and_iota(CNFWriter& w, int r, vector<int>& all_active) {
    // Chi (S-box) Layer
    for (int j = 0; j < 5; ++j) for (int k = 0; k < 64; ++k) {
        int act = v_act(r, j, k);
        all_active.push_back(act);
        vector<int> din(5), dout(5);
        for (int i = 0; i < 5; ++i) { din[i] = v_b(r, i, j, k); dout[i] = v_g(r, i, j, k); }

        for (uint32_t x = 0; x < 32; ++x) { // S-box valid differential transitions
            // (Validity logic omitted for brevity in listing, standard DDT check applies)
            // w.add_clause(clause);
        }
        w.add_clause({din[0], din[1], din[2], din[3], din[4], -act});
        for(int i = 0; i < 5; ++i) w.add_clause({-din[i], act});
    }
    // Iota Layer (Constant Addition)
    for (int i = 0; i < 5; ++i) for (int j = 0; j < 5; ++j) for (int k = 0; k < 64; ++k) {
        int bit_rc = (i == 0 && j == 0) ? ((RC[r] >> k) & 1) : 0;
        int v_out = v_a(r+1, i, j, k), v_in = v_g(r, i, j, k);
        w.add_clause({bit_rc ? -v_out : v_out, v_in});
        w.add_clause({bit_rc ? v_out : -v_out, -v_in});
    }
}

int main() {
    CNFWriter writer;
    vector<int> active_vars;

    // Build Round 1 & Round 2 (Full rounds)
    for(int r = 0; r <= 1; ++r) {
        build_linear_layer(writer, r);
        build_sbox_and_iota(writer, r, active_vars);
    }
```

```
    // Build Round 2.5 (Linear layer only)
    build_linear_layer(writer, 2);

    // ==========================================
    // Specialized Cryptanalytic Constraints
    // ==========================================

    // 1. SHA3-256 Constraints: Force difference in Capacity & Padding to 0
    // Capacity (512 bits) + Min Padding (4 bits) = 516 fixed bits
    for (int idx = 1084; idx < 1600; ++idx) {
        writer.add_clause({-(1 + idx)});
    }

    // 2. Prevent Trivial Solutions: Restrict initial active bits strictly within the Rate
    vector<int> non_zero;
    for (int i = 0; i < 5; ++i) for (int j = 0; j < 5; ++j) for (int k = 0; k < 64; ++k) {
        if (flat_id(i, j, k) < 1084) non_zero.push_back(v_a(0, i, j, k));
    }
    writer.add_clause(non_zero);

    // 3. Force alpha_1 into CP-kernel (Column Parity = 0 to minimize diffusion)
    for (int i = 0; i < 5; ++i) for (int k = 0; k < 64; ++k) {
        writer.add_clause({-v_cs(1, i, k)});
    }

    // 4. Differential Weight Restrictions
    vector<int> beta1_act;
    for (int j = 0; j < 5; ++j) for (int k = 0; k < 64; ++k) {
        beta1_act.push_back(v_act(1, j, k));
    }
    // Restrict active S-boxes in round 2 to <= 7
    add_weight_limit(writer, beta1_act, 7);
    // Restrict total active S-boxes across the trail to <= 20
    add_weight_limit(writer, active_vars, 20);

    writer.write("tida_trail.cnf");
    return 0;
}
```

D.2 Implementation of the Message Selection Phase

The following code listing demonstrates the core implementation of the Message Selection phase within the Target Internal Difference Algorithm (TIDA). In this phase, the non-linear constraints derived from the Values of Difference Conditions Table (VDCT) and the linear transformations (θ, ρ, π) of Keccak are systematically translated into an overdefined system of linear equations over GF(2). By employing Gaussian elimination, the algorithm calculates the rank of the algebraic system and identifies the available degrees of freedom (free variables). Subsequently, the system strictly enforces the boundary conditions, including the Capacity and Padding rules of SHA-3. Through an iterative process of randomized assignments to the free variables and continuous consistency checking, the program efficiently

filters out logical contradictions. Ultimately, this automated algebraic modeling successfully recovers the valid initial message space that perfectly conforms to the predetermined 2.5-round internal differential characteristic, establishing the mathematical foundation for the subsequent collision search.

Listing D.2: Message Selection Phase of the Target Internal Difference Algorithm (TIDA)

```
#include <iostream>
#include <vector>
#include <stdint.h>
#include <random>

using namespace std;

// Configuration for SHA3-384 Capacity bits
#define CAPACITY 99

typedef uint64_t u64;

// ... [Standard Keccak Round Constants (RC) and Difference Arrays (delta_in, delta_out) omitted for brevity] ...

// Construct algebraic equations from the Difference Conditions Table (DCT)
void IDTC(const u64 delta_in[25], const u64 delta_out[25]) {
    // ... [Matrix manipulations mapping S-box transition conditions omitted] ...
    // Converts DCT conditions into an overdefined linear system over GF(2)
}

// Apply Keccak's linear layer (theta, rho, pi) to the state matrices
void Transform(char In[1600][201], char Out[1600][201]) {
    // ... [Linear layer permutations omitted] ...
}

// Global algebraic system variables
int k1, k2, nx, fv, ne, nf;
char XA[1600][201], XB[1600][201], EQS[1600][201];

// Gaussian Elimination over GF(2)
// Returns the rank of the matrix, or -1 if the system is inconsistent (contradiction)
int Gaussian2(char D0[1600][201]) {
    int r = 0;
    int pivot_col[201];
    for(int i = 0; i < 201; i++) pivot_col[i] = -1;

    for (int c = 0; c < 200 && r < 1600; c++) {
        int sel = r;
        while (sel < 1600 && D0[sel][c] == 0) sel++;

        if (sel < 1600) {
            swap(D0[r], D0[sel]);
            pivot_col[r] = c;
            for (int i = 0; i < 1600; i++) {
                if (i != r && D0[i][c] == 1) {
                    for (int j = c; j < 201; j++) D0[i][j] ^= D0[r][j];
                }
            }
            r++;
        }
    }
    // Check for logical contradictions (e.g., 0 = 1)
    for (int i = r; i < 1600; i++) {
        if (D0[i][200] == 1) return -1;
```

```
    }
    return r;
}

// Enforce Padding rules and Capacity constraints
int setupeq(char A0[1600][201]) {
    for(int i = 0; i < fv; i++) for(int j = 0; j < 201; j++) EQS[i][j] = XA[i][j];
    for(int i = 0; i < k1; i++) for(int j = 0; j < 201; j++) EQS[fv+i][j] = A0[i][j];

    // Example for SHA3-384: Fixing capacity bits to 0
    if (CAPACITY == 99) {
        EQS[fv+k1][200] = 1;
        EQS[fv+k1+2][200] = 1;
        for(int i = 0; i < CAPACITY; i++) EQS[fv+k1+i][101+i] = 1;
    }
    return (fv + k1 + CAPACITY);
}

// Assign random values to free variables and check system consistency
int final(char A1[1600][201], unsigned char rand[10]) {
    // Assign random bits to the identified free variables
    for(int i = 0; i < fv; i++) EQS[i][200] = (rand[i/8] >> (i%8)) % 2;
    Gaussian2(EQS);

    // ... [Intermediate state backtracking and consistency checking omitted] ...

    // Return the final evaluation of the linear system
    return Gaussian2(EQS);
}

// Core execution: Find initial messages satisfying the 2-round internal differential
int internal_differential() {
    char A1[1600][201], A0[1600][201];
    unsigned char rand[10];
    std::mt19937 gen(std::random_device{}());
    std::uniform_int_distribution<int> distrib(0, 255);

    // 1. Establish algebraic constraints for Round 2
    IDTC(delta_in1, delta_out1); // Process DCT
    Transform(D_1, A1);          // Apply linear layer
    k2 = Gaussian2(A1);          // Solve and get rank

    // 2. Establish algebraic constraints for Round 1
    Transform(XB, XA);
    nx = Gaussian2(XA);
    IDTC(delta_in0, delta_out0);
    Transform(D_1, A0);
    k1 = Gaussian2(A0);

    fv = count_free(A0); // Calculate the Degrees of Freedom (DoF)

    // 3. Iteratively guess free variables until the system is consistent
    int nf;
    do {
        ne = setupeq(A0); // Apply boundary conditions (Capacity & Padding)
        for (int i = 0; i < 10; i++) rand[i] = distrib(gen);
        nf = final(A1, rand); // Evaluate system to detect contradictions
    } while (nf == -1); // Retry if 0 = 1 occurs

    return 0; // Valid initial message space successfully recovered
}

int main() {
    // Execute TIDA and retrieve the message candidates
```

```
    internal_differential();
    // ... [Verification via Keccak-f permutation omitted] ...
    return 0;
}
```